

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG



BÁO CÁO ĐỒ ÁN CHUYÊN NGÀNH

**TRIỂN KHAI HỆ THỐNG PHÁT HIỆN VÀ PHẢN ỨNG
SỰ CỐ BẢO MẬT DỰA TRÊN KIẾN TRÚC ZERO
TRUST CHO ỨNG DỤNG MICROSERVICES TRONG
MÔI TRƯỜNG MULTI-CLOUD**

*(Implementation of a Zero Trust-Based Security Detection and Response System for
Microservice Application in Multi-Cloud Environments)*

Ngành: An Toàn Thông Tin

Mã môn học: NT114.Q21.ANTT

Sinh viên thực hiện:

Hoàng Bảo Phước - 23521231

Phạm Võ Khánh Hà - 23520414

GVHD: ThS. Đỗ Thị Phương Uyên

TP. Hồ Chí Minh, tháng 6 năm 2026

LỜI CẢM ƠN

Trong suốt quá trình nghiên cứu và hoàn thành đề tài “Triển khai hệ thống phát hiện và ứng phó sự cố bảo mật dựa trên Zero Trust cho hệ thống hướng Microservices trên môi trường Multi-Cloud” thuộc học phần Đồ án chuyên ngành chương trình cử nhân An toàn thông tin, bên cạnh sự cố gắng, nỗ lực học hỏi không ngừng nghỉ, nhóm chúng em cũng đã nhận được rất nhiều sự hỗ trợ, góp ý và giúp đỡ quý báu từ quý thầy cô.

Trước tiên, với tất cả lòng biết ơn và sự kính trọng, nhóm xin được gửi lời tri ân chân thành và sâu sắc đến cô ThS. Đỗ Thị Phương Uyên – người đã trực tiếp hướng dẫn, tận tình hỗ trợ và đồng hành cùng chúng em trong suốt quá trình thực hiện đề tài. Những góp ý mang tính chuyên môn, những định hướng khoa học đúng đắn từ cô đã giúp nhóm chúng em có cơ sở vững chắc để phát triển tư duy nghiên cứu, hiểu sâu sắc hơn về mô hình Zero Trust trong môi trường thực tế. Bắt đầu từ những ngày đầu thực hiện đồ án cho đến nay, Cô đã luôn theo sát, tận tình hướng dẫn và đưa ra những góp ý thiết thực về mặt kỹ thuật. Sự hỗ trợ của cô đã giúp nhóm tháo gỡ nhiều vướng mắc trong quá trình triển khai hệ thống, hiện thực hóa các giải pháp công nghệ phức tạp lên môi trường giả lập sát với thực tế nhất.

Tiếp theo, nhóm xin trân trọng cảm ơn gia đình và các bạn sinh viên Khóa 18 - những người luôn là điểm tựa tinh thần vững vàng, là nguồn động lực to lớn để nhóm vượt qua khó khăn và giữ vững tinh thần học tập, nghiên cứu nghiêm túc.

Mặc dù nhóm đã cố gắng hoàn thiện đề tài một cách tốt nhất, nhưng với những giới hạn về thời gian, kiến thức và kinh nghiệm thực tiễn, đề tài chắc chắn không tránh khỏi những thiếu sót. Nhóm rất mong nhận được những ý kiến đóng góp từ cô để có thể cải thiện và phát triển hơn trong những nghiên cứu tiếp theo.

Một lần nữa, nhóm xin chân thành cảm ơn và kính chúc cô ThS. Đỗ Thị Phương Uyên luôn mạnh khỏe, hạnh phúc và đạt nhiều thành tựu trong sự nghiệp giảng dạy và nghiên cứu khoa học.

Nhóm xin chân thành cảm ơn!

TÓM TẮT

Đề tài xây dựng hệ thống Zero Trust - một nền tảng phát hiện và phản hồi bảo mật (Security Detection & Response) áp dụng kiến trúc Zero Trust cho ứng dụng microservices ngân hàng chạy đồng thời trên hai môi trường đám mây: AWS (public cloud) và OpenStack (private cloud). Vấn đề đặt ra là các hệ thống microservices truyền thống tin tưởng ngầm định lưu lượng nội bộ, khiến kẻ tấn công sau khi xâm nhập một thành phần có thể tấn công di chuyển ngang, trong khi các giải pháp bảo mật hiện có thiếu khả năng giám sát hợp nhất và phản hồi tự động trong môi trường đa đám mây.

Giải pháp đề xuất tích hợp năm lớp bảo vệ xuyên suốt vòng đời yêu cầu: WireGuard VPN mã hóa kênh truyền Layer 3 giữa hai cụm Kubernetes; SPIFFE/SPIRE cấp phát danh tính workload dưới dạng chứng chỉ X.509 SVID với TTL 1 giờ, loại bỏ thông tin xác thực tĩnh; Envoy sidecar chặn toàn bộ lưu lượng pod và thực thi mTLS; Open Policy Agent (OPA) kiểm tra JWT, SVID và vai trò người dùng trước mỗi yêu cầu; Keycloak OIDC quản lý xác thực người dùng với token RS256.

Hệ thống giám sát dựa trên bộ PLG (Promtail – Loki - Grafana): Promtail thu thập log Envoy và OPA từ mọi node, Loki lưu trữ tập trung với retention 30 ngày, Grafana chạy các truy vấn LogQL để phát hiện bất thường theo thời gian thực. AI Analyzer phân loại sự kiện theo ba mức độ nghiêm trọng (low/medium/high) với ba backend có thể hoán đổi: GPT-4o, Gemini, và bộ phân tích heuristic không cần API key. SOAR Engine tiếp nhận cảnh báo và thực thi playbook tự động (cô lập pod, thu hồi token, chặn IP) thông qua Kubernetes Python SDK.

Kết quả thực nghiệm với 11 kịch bản tấn công theo khung MITRE ATT&CK (brute force, JWT forgery, lateral movement, data exfiltration, v.v.) cho thấy: sáu kịch bản bị chặn tức thời ở tầng OPA/Envoy trước khi xâm nhập; bốn kịch bản được phát hiện qua cảnh báo Grafana trong dưới 60 giây; SOAR Engine khởi động phản hồi tự động trong vòng 90 giây kể từ khi phát sinh alert. Hệ thống đạt độ trễ P95 dưới 45 ms cho luồng thanh toán xuyên đám mây và duy trì hoạt động ổn định dưới tải 50 yêu cầu/giây.

Đóng góp chính: Mô hình triển khai Zero Trust thực tế kết hợp SPIRE + OPA + Envoy trên K3s đa đám mây; Kiến trúc SIEM nhẹ dựa trên log thay thế agent-based monitoring; Vòng khép kín phát hiện–phân tích AI–phản hồi tự động không cần can thiệp thủ công; Bộ kịch bản kiểm thử tái hiện được cho 11 kỹ thuật ATT&CK.

MỤC LỤC

LỜI CẢM ƠN.....	2
TÓM TẮT.....	3
MỤC LỤC	4
DANH MỤC HÌNH VẼ	8
DANH MỤC BẢNG	9
DANH MỤC CÁC TỪ VIẾT TẮT	10
CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI	11
1.1. Đặt vấn đề.....	11
1.1.1. Bối cảnh.....	11
1.1.2. Hạn chế của mô hình bảo mật truyền thống	11
1.1.3. Khoảng trống trong giải pháp hiện có	11
1.2. Mục tiêu nghiên cứu	12
1.2.1. Mục tiêu tổng quát	12
1.2.2. Mục tiêu cụ thể	12
1.3. Phạm vi và giới hạn	12
1.3.1. Phạm vi thực hiện	12
1.3.2. Giới hạn	13
1.4. Cấu trúc báo cáo	13
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT.....	14
2.1. Kiến trúc Zero Trust (Zero Trust Architecture - ZTA).....	14
2.1.1. Định nghĩa và nguyên tắc cốt lõi	14
2.1.2. Các thành phần ZTA theo NIST SP 800-207	14
2.1.3. ZTA trong bối cảnh Kubernetes Multi-Cloud.....	15
2.2. SPIFFE/SPIRE - Định danh Workload	15
2.2.1. SPIFFE (Secure Production Identity Framework for Everyone).....	15
2.2.2. SPIRE (SPIFFE Runtime Environment)	15
2.3. Open Policy Agent (OPA) - Policy Engine.....	16
2.4. Envoy Proxy - Policy Enforcement Point.....	16
2.5. Keycloak - Identity Provider.....	17

2.6. PLG Stack - SIEM tập trung.....	17
2.7. AI Analyzer - SOC Analyst tự động	17
2.8. SOAR Engine - Phản ứng tự động.....	18
2.9. Grafana Alerting - Cảnh báo theo ngưỡng.....	18
2.10. MITRE ATT&CK Framework.....	19
CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	20
3.1. Phân tích yêu cầu	20
3.1.1. Yêu cầu chức năng.....	20
3.1.2. Phân loại tài sản theo mức độ nhạy cảm.....	20
3.2. Kiến trúc tổng thể	21
3.3. Mô hình mạng Multi-Cloud.....	21
3.3.1. AWS VPC	21
3.3.2. OpenStack Network.....	21
3.3.3. WireGuard VPN - Kết nối liên cloud.....	22
3.4. Thiết kế luồng dữ liệu bảo mật	22
3.4.1. Ba lớp gateway	22
3.4.2. Luồng thanh toán cross-cloud đầy đủ	22
3.5. Thiết kế luồng phát hiện và phản ứng.....	23
CHƯƠNG 4: TRIỂN KHAI HỆ THỐNG	24
4.1. Hạ tầng Infrastructure as Code	24
4.1.1. Terraform - Provision Cloud Resources	24
4.1.2. Ansible - Configuration Management.....	24
4.2. SPIRE - Cấu hình định danh workload.....	25
4.2.1. Cấu hình SPIRE Server	25
4.2.2. SPIRE Agent và Workload Registration	25
4.3. Envoy - Cấu hình sidecar.....	26
4.3.1. Inbound Listener (Port 15006).....	26
4.3.2. Cross-Cloud Upstream Cluster	26
4.4. OPA - Policy as Code.....	27
4.4.1. Cây quyết định OPA	27
4.4.2. Policy chính - zta_policy.rego	27

4.5. Network Policy	28
4.6. Microservices - Ứng dụng tài chính	28
4.6.1. Stack kỹ thuật	28
4.6.2. Fraud Detection - Mô hình điểm số	29
4.6.3. API Gateway - Xác thực và Rate Limiting	29
4.7. PLG Stack - Pipeline thu thập log	29
4.7.1. Promtail Configuration	29
4.7.2. Grafana Dashboards.....	30
4.8. AI Analyzer	30
4.8.1. Polling Loop	30
4.8.2. LLM Prompt Engineering.....	31
4.8.3. HITL Flow	31
4.9. SOAR Engine	31
4.9.1. Mapping Attack → Playbook	31
4.9.2. Playbook Implementation - isolate_workload	31
4.9.3. RBAC Kubernetes cho SOAR.....	32
4.10. Grafana Alerting — Cấu hình cảnh báo và Email	32
CHƯƠNG 5: KIỂM THỬ VÀ ĐÁNH GIÁ	34
5.1. Phương pháp kiểm thử	34
5.1.1. Chiến lược kiểm thử	34
5.1.2. Môi trường kiểm thử.....	34
5.2. Kiểm thử luồng nghiệp vụ bình thường.....	34
5.2.1. Đăng nhập và xác thực.....	34
5.2.2. Chuyển tiền bình thường	34
5.3. Kiểm thử các kịch bản tấn công.....	35
5.3.1. Scenario 01 - Brute Force / JWT Failure	35
5.3.2. Scenario 02 - JWT Forgery.....	35
5.3.3. Scenario 03 - Lateral Movement / SVID Impersonation.....	36
5.3.4. Scenario 04 - Fraud Gate Bypass.....	36
5.3.5. Scenario 05 - High-Velocity Transaction.....	36
5.3.6. Scenario 06 - Data Exfiltration	36

5.3.7. Tổng hợp kết quả 20 kịch bản.....	36
5.4. Đánh giá hiệu năng	37
5.4.1. MTTD và MTTR	37
5.4.2. Overhead hiệu năng ZTA.....	38
5.4.3. Chi phí tài nguyên.....	38
5.5. So sánh với các giải pháp hiện có	38
CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	39
6.1. Kết luận.....	39
6.1.1. Tóm tắt đóng góp.....	39
6.1.2. Hạn chế	41
6.2. Hướng phát triển	42
6.2.1. Ngắn hạn (3–6 tháng)	42
6.2.2. Trung hạn (6–12 tháng)	42
6.2.3. Dài hạn (12+ tháng).....	43
TÀI LIỆU THAM KHẢO	44
PHỤ LỤC	45
Phụ lục A - Cấu hình môi trường	45
A.1. Yêu cầu cài đặt.....	45
A.2. Biến môi trường	45
A.3. Khởi động nhanh.....	46
Phụ lục B - Danh sách kịch bản kiểm thử.....	46
Phụ lục C - Prometheus Metrics	47
Phụ lục D - Grafana Alert Rules và Email cảnh báo	48
Phụ lục E - Luồng xử lý POST /payments chi tiết.....	48

DANH MỤC HÌNH VẼ

Hình	Tiêu đề
Hình 2.1	Mô hình Zero Trust Architecture theo NIST SP 800-207
Hình 2.2	Kiến trúc SPIFFE/SPIRE và vòng đời SVID
Hình 2.3	Luồng xử lý OPA ext authz trong Envoy
Hình 3.1	Kiến trúc tổng thể 4 lớp của đồ án
Hình 3.2	Sơ đồ mạng Multi-Cloud AWS và OpenStack
Hình 3.3	Ba lớp gateway trong hệ thống
Hình 3.4	Luồng thanh toán cross-cloud đầy đủ
Hình 3.5	Luồng phát hiện và phản ứng AI/SOAR/HITL
Hình 4.1	Sơ đồ hạ tầng Terraform - AWS VPC
Hình 4.2	Cấu trúc SPIRE Server và SPIRE Agent
Hình 4.3	Luồng xử lý request qua Envoy sidecar
Hình 4.4	Cây quyết định OPA policy
Hình 4.5	Pipeline log từ Promtail → Loki → AI Analyzer
Hình 4.6	Sơ đồ luồng HITL Approval
Hình 5.1	Grafana dashboard - ZTA Security Overview
Hình 5.2	Kết quả kịch bản Brute Force
Hình 5.3	Kết quả kịch bản JWT Forgery
Hình 5.4	Kết quả kịch bản Fraud Gate Bypass
Hình 5.5	Kết quả kịch bản Data Exfiltration
Hình 5.6	Biểu đồ phân phối thời gian phát hiện (MTTD)
Hình 5.7	Biểu đồ overhead hiệu năng ZTA
Hình 5.8	Email cảnh báo mẫu do Grafana Alerting gửi

DANH MỤC BẢNG

Bảng	Tiêu đề
Bảng 3.1	Phân loại tài sản theo mức độ nhạy cảm
Bảng 3.2	Ma trận phân bố workload theo cloud
Bảng 4.1	Cấu hình tài nguyên các node
Bảng 4.2	Danh sách SVID đã đăng ký
Bảng 4.3	Bảng phân loại các quy tắc OPA
Bảng 4.4	Mô hình điểm số fraud scoring
Bảng 4.5	Mapping attack type → playbook SOAR
Bảng 5.1	Tổng hợp kết quả 20 kịch bản kiểm thử
Bảng 5.2	Số liệu MTTD và MTTR
Bảng 5.3	So sánh ZTLab với các giải pháp hiện có

DANH MỤC CÁC TỪ VIẾT TẮT

Từ viết tắt	Tiếng Anh	Tiếng Việt
ZTA	Zero Trust Architecture	Kiến trúc Zero Trust
SIEM	Security Information and Event Management	Quản lý thông tin và sự kiện bảo mật
SOAR	Security Orchestration, Automation and Response	Điều phối, tự động hóa và phản ứng bảo mật
HITL	Human-in-the-Loop	Yếu tố con người trong quy trình kiểm soát
SPIFFE	Secure Production Identity Framework for Everyone	Khung định danh workload an toàn
SVID	SPIFFE Verifiable Identity Document	Tài liệu định danh có thể xác minh SPIFFE
OPA	Open Policy Agent	Tác nhân chính sách mở
PEP	Policy Enforcement Point	Điểm thực thi chính sách
PLG	Promtail-Loki-Grafana	Stack giám sát PLG
mTLS	Mutual TLS	TLS xác thực hai chiều
OIDC	OpenID Connect	Giao thức xác thực mở
JWT	JSON Web Token	Token xác thực JSON
IR	Incident Response	Phản ứng sự cố
MTTD	Mean Time to Detect	Thời gian phát hiện trung bình
MTTR	Mean Time to Respond	Thời gian phản ứng trung bình
IaC	Infrastructure as Code	Hạ tầng dưới dạng mã nguồn
K3s	Lightweight Kubernetes	Kubernetes siêu nhẹ
VPN	Virtual Private Network	Mạng riêng ảo

CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI

1.1. Đặt vấn đề

1.1.1. Bối cảnh

Trong kỷ nguyên chuyển đổi số, điện toán đám mây đã trở thành nền tảng công nghệ chủ đạo. Đặc biệt trong ngành Tài chính - Ngân hàng, mô hình Multi-Cloud ngày càng được áp dụng rộng rãi: hệ thống Core Banking vận hành trên Private Cloud để đảm bảo kiểm soát dữ liệu và tuân thủ quy định, trong khi các dịch vụ ngân hàng số, API Gateway và cổng thanh toán tận dụng Public Cloud để mở rộng linh hoạt theo nhu cầu.

Theo báo cáo Gartner (2024), hơn 85% doanh nghiệp hiện đang sử dụng ít nhất hai nhà cung cấp cloud khác nhau [1]. Cùng với đó, xu hướng microservices đã phân mảnh các ứng dụng truyền thống thành hàng chục đến hàng trăm service nhỏ, giao tiếp với nhau qua mạng, mở rộng đáng kể bề mặt tấn công (attack surface).

1.1.2. Hạn chế của mô hình bảo mật truyền thống

Các mô hình bảo mật truyền thống dựa trên network perimeter (tường lửa vùng biên mạng) và implicit trust (tin tưởng ngầm định theo phân đoạn mạng) đã chứng tỏ sự lỗi thời qua hàng loạt sự cố lớn:

- **SolarWinds (2020):** Kẻ tấn công vào được mạng nội bộ, di chuyển ngang mà không bị phát hiện trong nhiều tháng vì hệ thống tin tưởng traffic nội bộ.
- **Colonial Pipeline (2021):** Ransomware lây lan toàn bộ OT network qua VPN vì không có micro-segmentation.
- **Capital One (2019):** SSRF từ một EC2 instance cho phép đọc toàn bộ dữ liệu S3 bucket vì IAM role quá rộng.

Đặc điểm chung của các sự cố này là xuất hiện cơ chế tấn công lateral movement - một khi kẻ tấn công vào được mạng nội bộ, chúng có thể di chuyển tự do. Thiếu sự thống nhất về chính sách bảo mật và giám sát liên tục giữa các miền cloud khiến chỉ số MTTD (Mean Time to Detect) cao hơn trung bình 27% so với môi trường cloud đơn lẻ (IBM Cost of a Data Breach Report, 2023) [2].

1.1.3. Khoảng trống trong giải pháp hiện có

Thị trường đã có các giải pháp Zero Trust như Cloudflare One, Zscaler Private Access, Palo Alto Prisma Access. Tuy nhiên, các giải pháp này:

- Tập trung vào user-to-app access, chưa xử lý tốt workload-to-workload identity trong Kubernetes multi-cluster.
- Chi phí cao, cần phối hợp với nhiều bên liên quan để xây dựng, không phù hợp với nghiên cứu học thuật và doanh nghiệp vừa nhỏ.

- Thiếu SOAR tích hợp AI với cơ chế HITL phù hợp môi trường multi-cloud có phân tách dữ liệu nhạy cảm.
- Không hỗ trợ tốt việc tự động hóa phản ứng ở cấp độ Kubernetes workload (scale-to-zero, network isolation) mà vẫn đảm bảo có người duyệt trước khi hành động.

1.2. Mục tiêu nghiên cứu

1.2.1. Mục tiêu tổng quát

Nghiên cứu và triển khai mô hình phòng thủ chủ động dựa trên kiến trúc Zero Trust tích hợp SIEM/SOAR hỗ trợ AI trên hạ tầng Multi-Cloud sát với thực tế, nhằm tối ưu hóa MTTD và MTTR đồng thời đảm bảo có cơ chế kiểm soát của con người trước mọi quy trình vận hành tự động.

1.2.2. Mục tiêu cụ thể

STT	Mục tiêu	Thành phần đáp ứng
1	Loại bỏ implicit trust dựa trên IP; mọi workload phải có danh tính mã hóa xác minh được	SPIFFE/SPIRE
2	Thực thi kiểm soát truy cập theo nguyên tắc least privilege trên mọi service-to-service call	Envoy + OPA
3	Xác thực người dùng tập trung, hỗ trợ RBAC chi tiết theo vai trò tài chính	Keycloak OIDC
4	Thu thập và tập trung log từ cả hai cloud trong thời gian thực	PLG Stack
5	Tự động phân tích log, nhận diện kỹ thuật tấn công MITRE ATT&CK	AI Analyzer
6	Cảnh báo theo ngưỡng realtime và thông báo email tự động khi phát hiện bất thường	Grafana Alerting (alert rules + SMTP contact point)

1.3. Phạm vi và giới hạn

1.3.1. Phạm vi thực hiện

- **Hạ tầng:** Triển khai 2 K3s cluster tách biệt với AWS và OpenStack.
- **Ứng dụng:** 7 microservice mô phỏng hệ thống thanh toán ngân hàng, bao gồm API Gateway, Payment Service, Fraud Detection, Core Banking, Account Service, Transaction Service, Notification Service.
- **Bảo mật:** SPIFFE/SPIRE mTLS, Envoy ext_authz, OPA Policy-as-Code, Keycloak OIDC, Kubernetes NetworkPolicy, WireGuard VPN.
- **Thành phần giám sát liên tục:** PLG Stack, Prometheus custom metrics, AI Analyzer (OpenAI/Gemini/Heuristic), SOAR Engine.

- **Grafana Alerting:** alert rules LogQL, contact point SMTP email, notification policy.
 - **Kiểm thử:** 20 kịch bản tấn công từ scenario_01 đến scenario_20, covering MITRE ATT&CK Initial Access, Lateral Movement, Defense Evasion, Exfiltration, Impact.

1.3.2. Giới hạn

- Môi trường lab với tài nguyên hạn chế; chưa mô phỏng được traffic thực tế quy mô lớn (production scale).
- SPIRE Agent trên OpenStack dùng join_token attestation thay vì k8s_psat vì K3s single-node không hỗ trợ đầy đủ projected token.
- Grafana Alerting chỉ phát hiện bất thường theo ngưỡng khối lượng (threshold-based volume), chưa phân tích được ngữ nghĩa hay tương quan hành vi phức tạp.

1.4. Cấu trúc báo cáo

Báo cáo được tổ chức thành 6 chương:

- Chương 1 trình bày bối cảnh, vấn đề và mục tiêu nghiên cứu.
- Chương 2 trình bày cơ sở lý thuyết về các công nghệ được sử dụng.
- Chương 3 phân tích yêu cầu và đề xuất thiết kế kiến trúc hệ thống.
- Chương 4 mô tả chi tiết quá trình triển khai từng thành phần.
- Chương 5 trình bày kết quả kiểm thử và đánh giá toàn diện.
- Chương 6 tổng kết đóng góp và đề xuất hướng phát triển.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1. Kiến trúc Zero Trust (Zero Trust Architecture - ZTA)

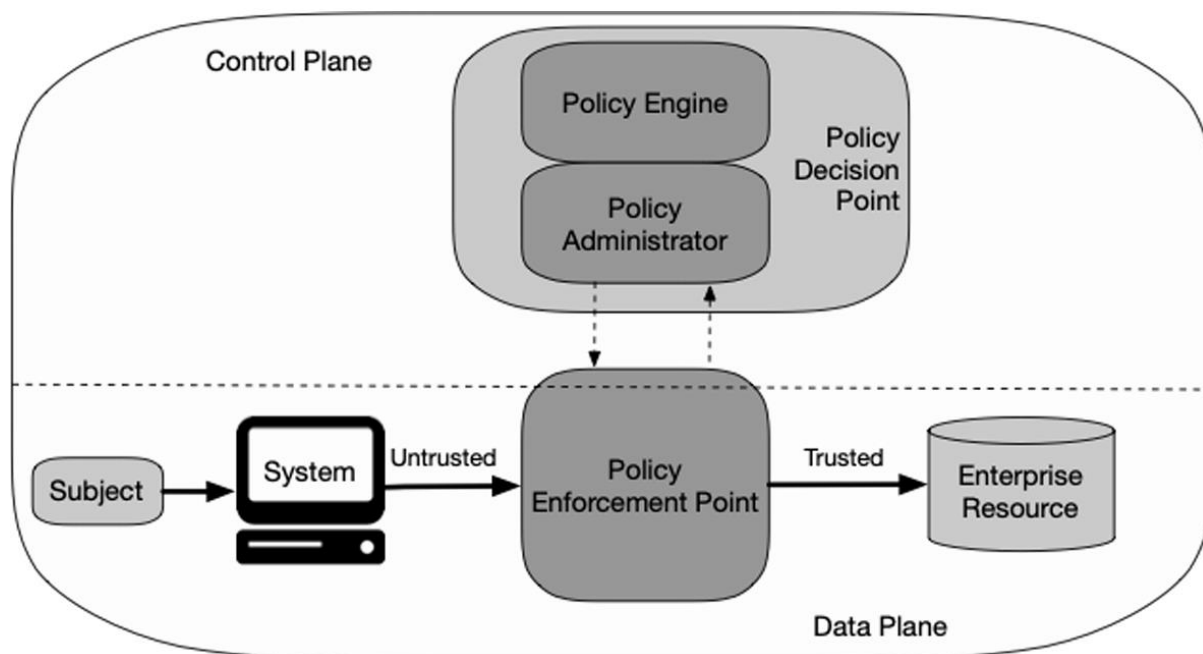
2.1.1. Định nghĩa và nguyên tắc cốt lõi

Zero Trust được NIST (Viện Tiêu chuẩn và Công nghệ Quốc gia Hoa Kỳ) định nghĩa trong tài liệu SP 800-207 [3] với nguyên lý cốt lõi: “Never Trust, Always Verify” - không bao giờ tin tưởng ngầm định, luôn xác minh liên tục.

Ba nguyên tắc nền tảng của ZTA:

- 1) **Verify Explicitly (Xác minh tường minh):** Mọi request truy cập tài nguyên đều phải được xác thực và ủy quyền dựa trên tất cả thông tin có sẵn: danh tính người dùng/ workload, thiết bị, vị trí, dịch vụ, dữ liệu và hành vi bất thường.
- 2) **Least Privilege Access (Quyền tối thiểu):** Giới hạn quyền truy cập của người dùng/workload ở mức tối thiểu cần thiết, áp dụng Just-In-Time (JIT) và Just-Enough-Access (JEA), đồng thời giảm thiểu thời gian giữ quyền.
- 3) **Assume Breach (Giả định đã bị xâm phạm):** Thiết kế hệ thống với giả định rằng kẻ tấn công đã vào trong mạng nội bộ, phải luôn đề phòng như mã hóa toàn bộ traffic, phân đoạn truy cập, giám sát liên tục và phát hiện bất thường.

2.1.2. Các thành phần ZTA theo NIST SP 800-207



Hình 2.1 - Mô hình Zero Trust Architecture theo NIST SP 800-207

2.1.3. ZTA trong bối cảnh Kubernetes Multi-Cloud

Triển khai ZTA trên Kubernetes vô cùng phức tạp, các pod có thể bị điều phối qua lại sang node, IP cấp cho pod chỉ tồn tại trong khoảng thời gian và không bền vững, còn service mesh thì có thể thay đổi topology mạng. Vì vậy, thay vì dựa vào địa chỉ IP, hệ thống cần xác thực workload thông qua danh tính (SVID) và thay các firewall rule tĩnh bằng chính sách truy cập được quản lý dưới dạng code bởi OPA.

2.2. SPIFFE/SPIRE - Định danh Workload

2.2.1. SPIFFE (Secure Production Identity Framework for Everyone)

SPIFFE là bộ tiêu chuẩn mã nguồn mở [4] định nghĩa cách cấp phát danh tính cho workload trong môi trường phân tán một cách tự động và an toàn. Danh tính SPIFFE có dạng URI:

```
spiffe://<trust-domain>/<path>
```

Ví dụ: `spiffe://ztlab.local/aws/payment-service`
`spiffe://ztlab.local/openstack/core-banking`

SVID (SPIFFE Verifiable Identity Document) là phần dữ liệu chứa danh tính SPIFFE, tồn tại ở hai dạng:

- **X.509-SVID:** Chứng chỉ TLS với SAN (Subject Alternative Name) chứa SPIFFE URI, dùng cho mTLS trên các kênh truyền.
- **JWT-SVID:** JWT token, dùng cho xác thực HTTP header.

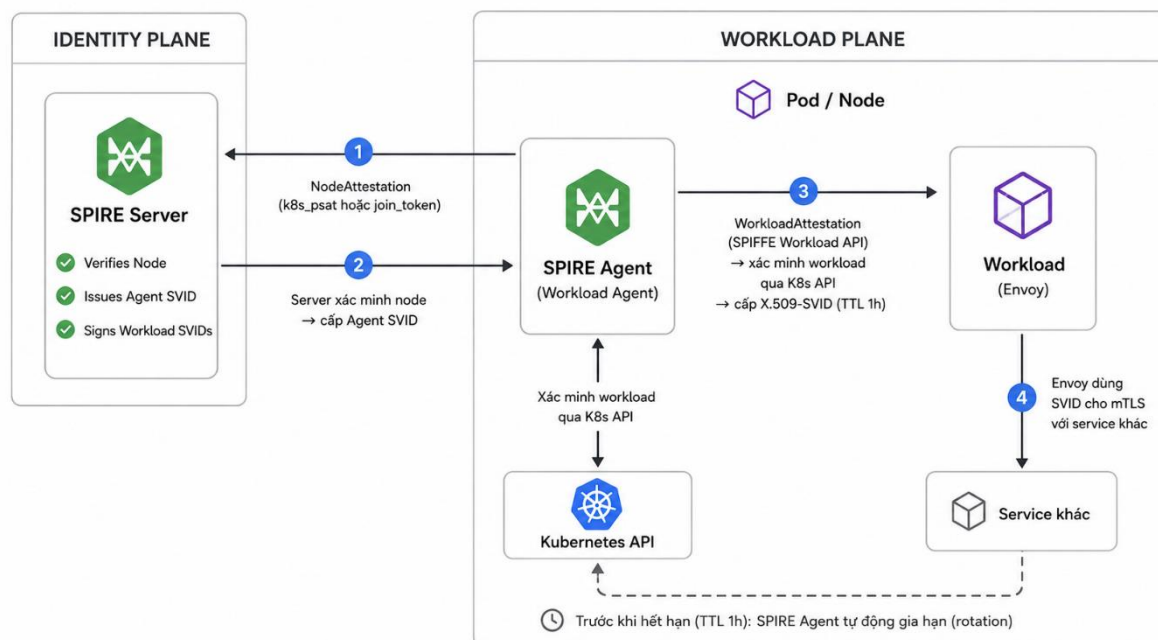
2.2.2. SPIRE (SPIFFE Runtime Environment)

SPIRE là phần mềm được tham chiếu dựa trên SPIFFE, gồm:

- **SPIRE Server:** CA nội bộ, quản lý trust bundle, chứng thực node và cấp SVID cho workload.
- **SPIRE Agent:** DaemonSet trên mỗi node, nhận attestation từ Server, cấp SVID cho workload thông qua Workload API (Unix socket).

Vòng đời SVID:

- 1) SPIRE Agent khởi động → NodeAttestation (k8s_psat hoặc join_token)
- 2) Server xác minh node → cấp Agent SVID
- 3) Workload (Envoy) mở kết nối đến Workload API
- 4) Agent WorkloadAttestation → xác minh workload qua K8s API
- 5) Agent cấp X.509-SVID (TTL 1h) cho workload
- 6) Envoy dùng SVID cho mTLS với service khác
- 7) Trước khi hết hạn: Agent tự động gia hạn (key rotation)

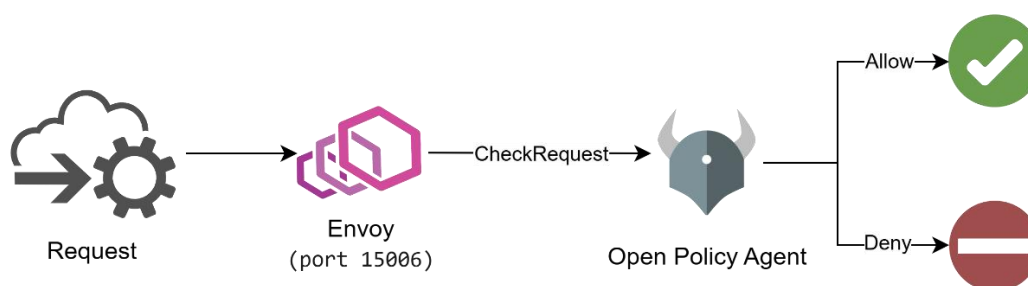


[Hình 2.2 - Kiến trúc SPIFFE/SPIRE và vòng đời SVID]

2.3. Open Policy Agent (OPA) - Policy Engine

OPA [5] là policy engine Policy as Code, chính sách policy viết bằng ngôn ngữ Rego được đánh giá và kiểm tra tại môi trường runtime, không cần biên dịch lại ứng dụng. OPA nhận input là ngữ cảnh request và trả về decision (boolean hoặc JSON).

Tích hợp OPA với Envoy, Envoy hỗ trợ giao thức ext_authz (External Authorization) theo chuẩn gRPC. Mỗi request vào pod đều được Envoy gửi một CheckRequest sang OPA trước khi được chuyển tiếp:



[Hình 2.3 - Luồng xử lý OPA ext_authz trong Envoy]

2.4. Envoy Proxy - Policy Enforcement Point

Envoy [6] là proxy L7 hiệu năng cao, viết bằng C++. Trong kiến trúc Zero Trust, Envoy chạy như **sidecar container** trong pod, đóng vai trò PEP - điểm duy nhất mà mọi traffic phải đi qua.

Cấu hình tính năng của Envoy được triển khai cụ thể như sau:

- Listener 15006: Cổng xử lý inbound traffic, gọi OPA ext_authz.
- Listener 15001: Cổng xử lý outbound traffic từ ứng dụng, route theo đường dẫn sang upstream service.
- SDS (Secret Discovery Service): Cơ chế tích hợp SPIRE Agent để lấy SVID/certificate tự động, không cần file tĩnh.
- Access Log: Tính năng ghi log với JSON format, ghi mọi request với trace_id, source SVID, response code, latency.

2.5. Keycloak - Identity Provider

Keycloak [7] là Identity Provider mã nguồn mở hỗ trợ OIDC/OAuth2/SAML. Trong đồ án triển khai lần này, Keycloak đóng vai trò quản lý danh tính người dùng cuối (end-user identity):

- Realm 'ztlab': Đơn vị định danh độc lập chứa tập người dùng, client, chính sách.
- Client 'web-portal' và 'api-gateway': Hai đơn vị đăng ký làm OAuth2 client để yêu cầu và nhận token xác thực.
- JWT RS256: Token được ký bằng RSA private key, verify bằng JWKS endpoint công khai.
- Claim 'realm_access.roles': Chứa danh sách role của user (financial-read, financial-write, security-analyst).

2.6. PLG Stack - SIEM tập trung

Promtail-Loki-Grafana (PLG) là stack giám sát nhẹ với ưu điểm tiêu thụ ít bộ nhớ hơn khoảng 10 lần:

Thành phần	Vai trò	Ghi chú
Promtail	Log collector/shipper	DaemonSet, đọc từ /var/log/pods/*
Loki	Log aggregation backend	Index theo label, không full-text
Grafana	Visualization + Alerting	Query Loki bằng LogQL

- LogQL là ngôn ngữ query của Loki, tương tự PromQL:

```
{job="kubernetes-pods", namespace="financial", app="payment-service"}
|="fraud_gate_bypass"
|json
|line_format "{{.trace_id}} - {{.event}}"
```

2.7. AI Analyzer - SOC Analyst tự động

AI Analyzer đóng vai trò **Tier-1 SOC Analyst** tự động, thực hiện 3 nhiệm vụ:

Thu thập log từ Loki định kỳ (mỗi 30 – 120 giây).

Phân tích ngữ nghĩa bằng LLM hoặc heuristic patterns.

Tạo alert có cấu trúc với MITRE ATT&CK mapping và recommended playbook.

Provider chain:

OpenAI GPT-4o-mini → (nếu lỗi/hết quota) → Gemini 1.5 Flash → (fallback) → Heuristic rules

Output schema chuẩn:

```
{
  "verdict": "malicious",
  "severity": "high",
  "confidence": 0.92,
  "attack_type": "brute_force",
  "attack_techniques": ["T1110.001", "T1078"],
  "recommended_playbook": "revoke_user_sessions",
  "affected_service": "api-gateway",
  "source_ip": "10.0.0.99",
  "reasoning": "Phát hiện 47 lần 401 từ cùng IP trong 2 phút"
}
```

2.8. SOAR Engine - Phản ứng tự động

SOAR Engine nhận alert đã được admin phê duyệt và thực thi **playbook** - chuỗi hành động tự động thông qua Kubernetes Python SDK và Keycloak Admin API:

Playbook	Cơ chế	Mục đích
isolate_workload	Patch Service selector với label không tồn tại	Ngắt traffic vào pod, giữ pod cho forensics
restrict_egress	Patch NetworkPolicy chặn toàn bộ egress	Ngăn data exfiltration tiếp diễn
quarantine_workload	Scale Deployment replicas=0	Tắt hoàn toàn service bị compromise
block_source_ip	Tạo NetworkPolicy chặn source IP cụ thể	Chặn IP tấn công
revoke_user_sessions	Gọi Keycloak Admin API logout-all	Hủy session user đang bị tấn công
monitor_only	Chỉ ghi log, không hành động	Alert cần theo dõi thêm

Cơ chế **HITL (Human-in-the-Loop)**: Alert có severity high hoặc critical vào hàng đợi pending, chờ admin phê duyệt qua Web Portal hoặc Grafana trước khi SOAR thực thi.

2.9. Grafana Alerting - Cảnh báo theo ngưỡng

Grafana Alerting [8] là hệ thống cảnh báo tích hợp sẵn trong Grafana, cho phép định nghĩa và quản lý cảnh báo dựa trên truy vấn dữ liệu giám sát, gồm các thành phần chính:

- **Alert Rules:** định nghĩa điều kiện cảnh báo bằng truy vấn LogQL trên Loki, kích hoạt khi giá trị vượt ngưỡng (threshold) trong một khoảng thời gian xác định.
- **Contact Points:** điểm tiếp nhận thông báo; ZTLab sử dụng contact point loại SMTP email để gửi cảnh báo trực tiếp tới quản trị viên.
- **Notification Policies:** định tuyến (routing) cảnh báo tới contact point phù hợp dựa trên nhãn severity và service của alert.

- **Cơ chế trạng thái (state machine):** alert rule chuyển từ Normal sang Pending khi điều kiện chớm vượt ngưỡng, và sang Firing khi điều kiện duy trì quá thời gian "for"; thông báo chỉ được gửi khi alert ở trạng thái Firing nhằm giảm cảnh báo nhiễu.

Trong ZTLab, Grafana Alerting hoạt động song song với AI Analyzer: các alert rule LogQL liên tục đánh giá log trong Loki, và khi phát hiện vượt ngưỡng (ví dụ số lần xác thực thất bại hay số giao dịch bị chặn) sẽ gửi email cảnh báo trực tiếp tới quản trị viên mà không phụ thuộc vào công cụ quản lý sự cố bên ngoài.

2.10. MITRE ATT&CK Framework

MITRE ATT&CK [9] là cơ sở tri thức về kỹ thuật tấn công của adversary, được tổ chức thành **Tactics** → **Techniques** → **Sub-techniques**. ZTLab mapping 20 kịch bản kiểm thử vào ATT&CK:

Tactic	Kịch bản	Technique
Initial Access	Credential Stuffing	T1078.003
Credential Access	Brute Force	T1110.001
Defense Evasion	JWT Forgery	T1550.001
Lateral Movement	SVID Impersonation	T1550.001
Exfiltration	Large Response	T1030
Impact	Cryptomining	T1496
Privilege Escalation	Privilege Escalation	T1068

CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Phân tích yêu cầu

3.1.1. Yêu cầu chức năng

Nhóm yêu cầu ứng dụng (Application Requirements):

- Hệ thống tài chính phải hỗ trợ luồng chuyển tiền cross-cloud (AWS → OpenStack).
- Xác thực người dùng qua Keycloak OIDC, phân quyền theo role.
- Fraud detection realtime với threshold có thể cấu hình.
- Giao diện web để người dùng chuyển tiền, xem số dư, lịch sử giao dịch.
- Giao diện quản trị để admin xem alerts và phê duyệt SOAR action.

Nhóm yêu cầu bảo mật (Security Requirements):

- Mọi workload-to-workload call phải có mTLS với SVID.
- Mọi user request phải có JWT hợp lệ từ Keycloak.
- Policy kiểm soát truy cập phải là code, version-controlled, kiểm tra được.
- Log từ mọi thành phần phải được tập trung về một hệ thống SIEM duy nhất.
- Phát hiện tấn công phải xảy ra trong vòng dưới 5 phút.
- Mọi hành động SOAR phải được admin phê duyệt (HITL) trước khi thực thi.
- Có cơ chế rollback mọi hành động SOAR.

Nhóm yêu cầu vận hành (Operational Requirements):

- Triển khai bằng IaC (Terraform + Ansible), có thể tái tạo từ đầu.
- Hệ thống giám sát hiển thị trạng thái real-time.
- Hỗ trợ dry_run mode để test playbook mà không tác động production.
- Audit trail đầy đủ, persistent qua các lần restart.

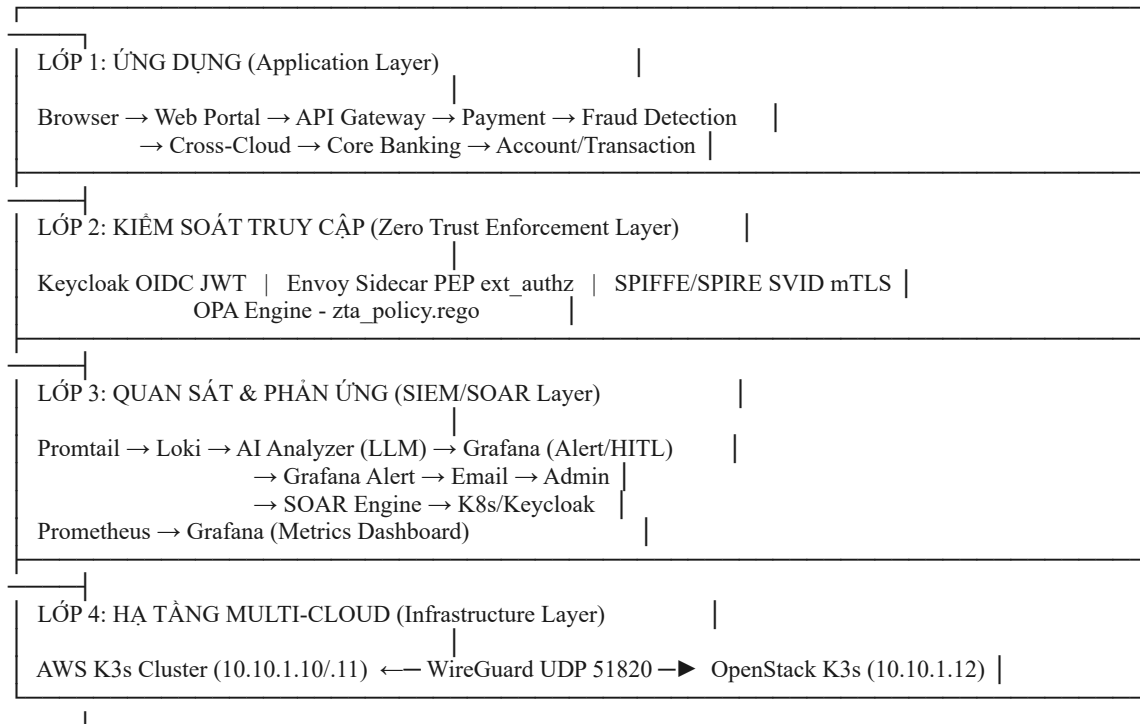
3.1.2. Phân loại tài sản theo mức độ nhạy cảm

[Bảng 3.1 - Phân loại tài sản theo mức độ nhạy cảm]

Tài sản	Loại	Mức nhạy cảm	Vị trí
Số dư tài khoản	Data	Tối mật	OpenStack PostgreSQL
Lịch sử giao dịch	Data	Tối mật	OpenStack PostgreSQL
JWT Access Token	Credential	Tối mật	In-memory (Redis)
SVID X.509	Credential	Bí mật	SPIRE Agent (Unix socket)
OPA Policy	Configuration	Nội bộ	K8s ConfigMap
Application Log	Operational	Nội bộ	Loki (AWS)
Keycloak DB	Data	Tối mật	K8s PVC (AWS)

3.2. Kiến trúc tổng thể

Hệ thống ZTLab được thiết kế theo mô hình **4 lớp**:

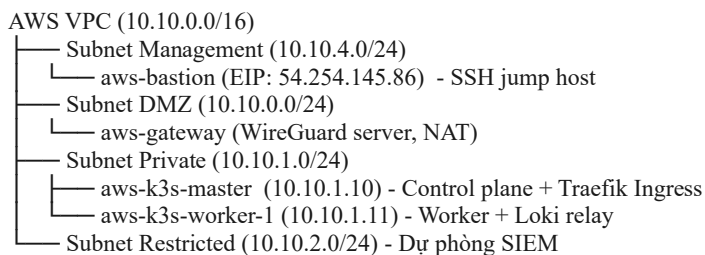


[Hình 3.1 - Kiến trúc tổng thể 4 lớp của ZTLab]

(Chèn hình vẽ kiến trúc đầy đủ)

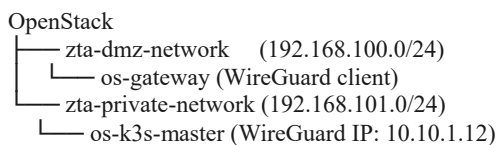
3.3. Mô hình mạng Multi-Cloud

3.3.1. AWS VPC



Không có instance nào trong Private subnet có Public IP. Mọi truy cập từ internet phải qua bastion (SSH tunnel) hoặc WireGuard.

3.3.2. OpenStack Network



os-k3s-master nhận IP WireGuard 10.10.1.12 cùng dải AWS - cho phép Envoy trên AWS định tuyến cross-cloud mà không cần DNS cross-cluster.

3.3.3. WireGuard VPN - Kết nối liên cloud

[aws-gateway: 10.10.0.10] ← ChaCha20-Poly1305 UDP 51820 → [os-gateway]
Route: 10.10.1.0/24 Route: 10.10.1.0/24

Traffic cross-cloud (payment-service → core-banking) đi qua WireGuard - mã hóa tại tầng mạng, bổ sung thêm một lớp ngoài mTLS ứng dụng.

[Hình 3.2 - Sơ đồ mạng Multi-Cloud AWS và OpenStack]

(Chèn network diagram)

3.4. Thiết kế luồng dữ liệu bảo mật

3.4.1. Ba lớp gateway

Một điểm thiết kế quan trọng cần phân biệt rõ là hệ thống có **ba lớp gateway khác nhau**, mỗi lớp có chức năng và vị trí khác nhau:

[Hình 3.3 - Ba lớp gateway trong hệ thống]

(Chèn hình vẽ 3 lớp gateway)

Lớp	Thành phần	Input	Output	Chức năng bảo mật
Infrastructure Gateway	SSH Tunnel, WireGuard, Traefik Ingress	HTTP từ Internet	Request vào K8s Service	Mã hóa mạng, NAT, routing theo Host header
API Gateway	services/api-gateway	Bearer JWT + JSON body	Request có trace_id + user context	JWT verify, role check, rate limit
Envoy Sidecar Gateway	envoy/configmap.yaml	HTTP (plain hoặc mTLS)	Forward hoặc Deny	OPA ext_authz, mTLS SVID, access log

3.4.2. Luồng thanh toán cross-cloud đầy đủ

[Hình 3.4 - Luồng thanh toán cross-cloud đầy đủ]

(Chèn sequence diagram 7 bước)

Tóm tắt 7 bước:

[Browser → Web Portal] Đăng nhập, lấy JWT từ Keycloak, gọi /api/transfer.

[Web Portal → API Gateway] Forward với Bearer JWT, Traefik route theo Host header.

[API Gateway] Verify JWT, kiểm tra role financial-write, tạo X-Trace-ID, forward sang Payment.

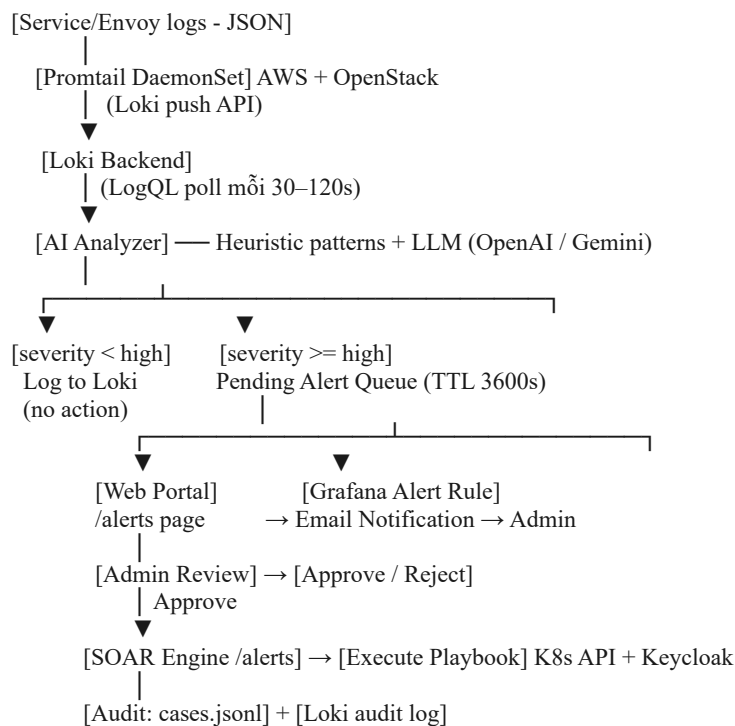
[Envoy → OPA → Payment Service] Envoy intercept, OPA quyết định allow (SVID hợp lệ hoặc Bearer + role đúng).

[Payment → Fraud Detection] Tính fraud score từ Redis velocity, amount, channel, country.

[Payment → Core Banking cross-cloud] Envoy outbound route /transactions/execute → 10.10.1.12:30081 → OpenStack Envoy → OPA → core-banking.

[Core Banking → Account + Transaction Service] Trừ/cộng số dư PostgreSQL, ghi ledger; response trả ngược về chain.

3.5. Thiết kế luồng phát hiện và phản ứng



[Hình 3.5 - Luồng phát hiện và phản ứng AI/SOAR/HITL]

(Chèn flowchart)

CHƯƠNG 4: TRIỂN KHAI HỆ THỐNG

4.1. Hạ tầng Infrastructure as Code

4.1.1. Terraform - Provision Cloud Resources

Toàn bộ hạ tầng cloud được định nghĩa bằng Terraform, đảm bảo tái tạo được (reproducible) và version-controlled:

AWS (terraform/aws/):

- VPC với 4 subnet phân vùng (Management, DMZ, Private, Restricted).
- Security Group chỉ mở port 6443 (K8s API), 8080 (HTTP), 30000–32767 (NodePort) trong nội bộ VPC; port 22 chỉ từ IP admin.
- EC2 instances t3.medium với Ubuntu 22.04 LTS.
- Elastic IP cho bastion.
- S3 bucket cho Terraform state + DynamoDB lock table.

OpenStack (terraform/openstack/):

- 2 network (dmz + private) với router kết nối.
- Security Group tương tự AWS.
- Nova instances với flavor phù hợp.

[Hình 4.1 - Sơ đồ hạ tầng Terraform - AWS VPC]

(Chèn hình vẽ VPC architecture)

4.1.2. Ansible - Configuration Management

Sau khi Terraform tạo VM, Ansible cấu hình:

- baseline.yml: Cài Docker, K3s, các dependency (kubectl, helm, jq), hardening OS cơ bản.
- wireguard.yml: Sinh WireGuard keypair, cấu hình wg0 interface và routing table trên cả hai phía.
- promtail.yml: Deploy Promtail với config gán label cloud=aws|openstack.

K3s được chọn thay kubeadm vì: binary ~70MB, built-in local-path provisioner cho PVC, phù hợp EC2 t3.medium (2 vCPU, 4GB RAM).

[Bảng 4.1 - Cấu hình tài nguyên các node]

Node	Instance	vCPU	RAM	Disk	Vai trò
aws-bastion	t3.micro	2	1GB	8GB	SSH jump host
aws-k3s-master	t3.medium	2	4GB	30GB	K3s control plane, Traefik
aws-k3s-worker-1	t3.medium	2	4GB	30GB	K3s worker, Loki relay

Node	Instance	vCPU	RAM	Disk	Vai trò
os-k3s-master	vm.medium	2	4GB	40GB	K3s standalone (OpenStack)

4.2. SPIRE - Cấu hình định danh workload

4.2.1. Cấu hình SPIRE Server

SPIRE Server chạy trong namespace identity trên AWS cluster:

```
# spire/server-config.yaml (trích)
server:
  trust_domain: "ztlab.local"
  ca_ttl: "168h"    # CA certificate TTL 7 ngày
  svid_ttl: "1h"    # Workload SVID TTL 1 giờ

node_attestor:
  k8s_psat:        # Projected Service Account Token (AWS)
  clusters:
    - ztlab-aws
  join_token:      # Cho OpenStack agent
  enabled: true
```

4.2.2. SPIRE Agent và Workload Registration

SPIRE Agent chạy như DaemonSet trên mỗi node. Mỗi workload phải được đăng ký với SPIRE Server:

[Bảng 4.2 - Danh sách SVID đã đăng ký]

SPIFFE URI	Selector	Cloud
spiffe://ztlab.local/aws/api-gateway	k8s:pod-label:app:api-gateway	AWS
spiffe://ztlab.local/aws/payment-service	k8s:pod-label:app:payment-service	AWS
spiffe://ztlab.local/aws/fraud-detection	k8s:pod-label:app:fraud-detection	AWS
spiffe://ztlab.local/aws/notification-service	k8s:pod-label:app:notification-service	AWS
spiffe://ztlab.local/openstack/core-banking	k8s:pod-label:app:core-banking	OpenStack
spiffe://ztlab.local/openstack/account-service	k8s:pod-label:app:account-service	OpenStack
spiffe://ztlab.local/openstack/transaction-service	k8s:pod-label:app:transaction-service	OpenStack

SPIRE Agent OpenStack kết nối về SPIRE Server trên AWS qua NodePort 30900, đường đi qua WireGuard tunnel.

[Hình 4.2 - Cấu trúc SPIRE Server và SPIRE Agent]

(Chèn hình vẽ SPIRE deployment)

4.3. Envoy - Cấu hình sidecar

Cấu hình Envoy được lưu trong envoy/configmap.yaml và mount vào các pod tài chính:

4.3.1. Inbound Listener (Port 15006)

```
listeners:
  - name: inbound_15006
    address:
      socket_address: {address: 0.0.0.0, port_value: 15006}
    filter_chains:
      - filters:
          - name: envoy.filters.network.http_connection_manager
            typed_config:
              stat_prefix: inbound
              http_filters:
                - name: envoy.filters.http.ext_authz
                  typed_config:
                    grpc_service:
                      envoy_grpc:
                        cluster_name: opa_authz
                      failure_mode_allow: false # DENY nếu OPA không phản hồi
                - name: envoy.filters.http.router
            route_config:
              virtual_hosts:
                - name: local_service
                  domains: ["*"]
                  routes:
                    - match: {prefix: "/"}
                      route: {cluster: local_service}
```

Lưu ý failure_mode_allow: false - nếu OPA server không khả dụng, Envoy từ chối mọi request, không cho phép ngầm định.

4.3.2. Cross-Cloud Upstream Cluster

```
clusters:
  - name: core_banking
    type: STATIC # IP tĩnh, không dùng DNS
    connect_timeout: 8s
    load_assignment:
      endpoints:
        - lb_endpoints:
            - endpoint:
                address:
                  socket_address:
                    address: 10.10.1.12 # OpenStack WireGuard IP
                    port_value: 30081 # NodePort core-banking
    transport_socket:
      name: envoy.transport_sockets.tls
      typed_config:
        combined_validation_context:
          default_validation_context:
            match_typed_subject_alt_names:
              - san_type: URI
            matcher:
              exact: "spiffe://ztlab.local/openstack/core-banking"
```

Cross-cloud mTLS: Envoy của payment-service dùng SVID của payment-service để xác thực với Envoy của core-banking, và yêu cầu core-banking phải có SVID spiffe://ztlab.local/openstack/core-banking.

[Hình 4.3 - Luồng xử lý request qua Envoy sidecar]

(Chèn hình sequence diagram Envoy→OPA→App)

4.4. OPA - Policy as Code

4.4.1. Cây quyết định OPA

[Hình 4.4 - Cây quyết định OPA policy]

(Chèn decision tree)

4.4.2. Policy chính - zta_policy.rego

```
package zta.authz

import future.keywords.if
import future.keywords.in

default allow = false

# Public health endpoints
allow if {
  path in ["/health", "/ready", "/metrics"]
}

# External user request - có JWT Bearer, không có SVID
allow if {
  external_api_request
}

# Internal service call - có SVID từ trust domain
allow if {
  internal_service_request
}

# Core transaction - SVID + fraud gate đã pass
allow if {
  core_transaction_with_fraud_gate
}

external_api_request if {
  method == "POST"
  path == "/payments"
  has_bearer_token
  not valid_svid
}

internal_service_request if {
  valid_svid
  svid_in_trust_domain
  method in {"GET", "POST", "OPTIONS"}
  startswith(path, "/payments")
}

core_transaction_with_fraud_gate if {
  valid_svid
  svid_in_trust_domain
  method == "POST"
  path == "/transactions/execute"
  headers["x-fraud-gate"] == "passed"
  to_number(headers["x-fraud-score"]) < 75
}
```

[Bảng 4.3 - Bảng phân loại các quy tắc OPA]

Rule	Điều kiện	Kết quả
public_path	path ∈ {/health, /ready, /metrics}	ALLOW
external_api_request	Bearer JWT hợp lệ, role đúng, không có SVID	ALLOW
internal_service_request	SVID hợp lệ từ trust domain ztlab.local	ALLOW
core_transaction_with_fraud_gate	SVID + X-Fraud-Gate:passed + score < 75	ALLOW
Default	Tất cả trường hợp còn lại	DENY

4.5. Network Policy

Kubernetes NetworkPolicy giới hạn traffic ở tầng IP/Port, bổ sung thêm một lớp bảo vệ ngoài Envoy/OPA:

AWS Cluster:

- Financial pods: nhận ingress từ namespace financial, monitoring, kube-system, identity.
- Egress: DNS (53), internal cluster, identity namespace.
- **Cross-cloud egress rule:** chỉ cho phép 10.10.1.12/32:30081 - không có pod AWS nào ngoài payment-service có thể gọi sang OpenStack.

OpenStack Cluster:

- Financial pods: nhận ingress từ internal cluster + subnet 10.10.1.0/24 (WireGuard dải AWS).
- Egress: DNS, intra-cluster (8080, 5432), Loki relay 10.10.1.11:31100.

4.6. Microservices - Ứng dụng tài chính

4.6.1. Stack kỹ thuật

Tất cả service viết bằng **Python 3.11 + FastAPI + uvicorn**. Có sử dụng thư viện chung trong shared/:

- shared/logging.py: ZTLabLogger - structured JSON logging với fields chuẩn {timestamp, level, service, cloud, trace_id, event}.
- shared/metrics.py: Prometheus custom metrics đã định nghĩa sẵn.

Build image từ một Dockerfile dùng build arg SERVICE_NAME:

```
FROM python:3.11-slim
WORKDIR /app
ARG SERVICE_NAME
COPY services/${SERVICE_NAME}/requirements.txt ./
RUN pip install --no-cache-dir -r requirements.txt
COPY shared/.shared/
COPY services/${SERVICE_NAME}/ ./
```

EXPOSE 8080
CMD ["python", "main.py"]

4.6.2. Fraud Detection - Mô hình điểm số

[Bảng 4.4 - Mô hình điểm số fraud scoring]

Yếu tố	Điều kiện	Điểm cộng
Baseline	Mọi giao dịch	+5
Velocity trung bình	Count > LIMIT/2 trong 60s	+10
Velocity cao	Count > LIMIT trong 60s	+25
Velocity rất cao	Count > LIMIT×3 trong 60s	+40
High amount	amount ≥ 100,000,000 VND	+30
Critical amount	amount ≥ 500,000,000 VND	+55
Risky channel	tor, unknown, script	+15
Unusual country	Ngoài {VN, SG, TH}	+10

Ngưỡng quyết định:

Score	Verdict	Gate	Hành động
< 40	allow	passed	Tiếp tục xử lý
40 – 74	review	passed	Tiếp tục + cảnh báo
≥ 75	block	blocked	Từ chối giao dịch

Core Banking validate lại fraud gate header - chống bypass header injection.

4.6.3. API Gateway - Xác thực và Rate Limiting

```
# Trích api-gateway/main.py
async def verify_token(token: str) -> dict:
    # Ưu tiên 1: RS256 từ Keycloak JWKS
    try:
        jwks = await get_keycloak_jwks() # cached
        payload = jwt.decode(token, jwks, algorithms=["RS256"],
                               issuer=KEYCLOAK_ISSUER)
        return payload
    except Exception:
        pass
    # Ưu tiên 2: HS256 dev token (chỉ khi có JWT_DEV_SECRET)
    if JWT_DEV_SECRET:
        payload = jwt.decode(token, JWT_DEV_SECRET, algorithms=["HS256"])
        return payload
    raise HTTPException(401, "invalid token")
```

Rate limiting: in-memory counter per source IP, window 60 giây, threshold 60 request.

4.7. PLG Stack - Pipeline thu thập log

4.7.1. Promtail Configuration

```
# ansible/promtail-config.yaml (trích)
scrape_configs:
  - job_name: kubernetes-pods
    kubernetes_sd_configs:
      - role: pod
    relabel_configs:
      - source_labels: [__meta_kubernetes_namespace]
```

```
target_label: namespace
- source_labels: [__meta_kubernetes_pod_label_app]
  target_label: app
- target_label: cloud
  replacement: aws # hoặc openstack - inject tại deploy time
```

Relay cho OpenStack: Promtail trên OpenStack đẩy log về http://10.10.1.11:31100 - socat relay trên aws-worker-1 - sau đó forward vào Loki ClusterIP nội bộ. Điều này giúp OpenStack cluster không cần kết nối trực tiếp vào Loki service port.

4.7.2. Grafana Dashboards

Hệ thống có 5 Grafana dashboard:

ZTA Security Overview: Tổng quan OPA decisions (allow/deny rate), JWT failures, fraud blocks, SOAR cases active.

Envoy Access Logs: HTTP matrix, response time P50/P95/P99, error rate theo service.

OPA Decision Log: Allow/deny rate theo service, top denied paths, denied reason breakdown.

AI SIEM SOAR: AI verdicts distribution, attack types, SOAR cases, Grafana alert firing count.

Threat Intelligence Feed: MITRE ATT&CK heatmap, top source IPs, verdict distribution, live event stream.

[Hình 5.1 - Grafana dashboard - ZTA Security Overview]

(Chèn screenshot Grafana)

4.8. AI Analyzer

4.8.1. Polling Loop

AI Analyzer chạy background task poll Loki theo chu kỳ:

```
async def poll_loop():
    while True:
        logs = await fetch_loki_logs(
            query=LOKI_QUERY,
            start=now - LOOKBACK_SECONDS,
            limit=MAX_LOGS_PER_BATCH
        )
        if logs:
            result = await analyze_batch(logs)
            if result.severity >= MIN_ALERT_SEVERITY:
                await handle_alert(result)
            await asyncio.sleep(POLL_INTERVAL_SECONDS)
```

4.8.2. LLM Prompt Engineering

Khi dùng OpenAI/Gemini, AI Analyzer gửi prompt có cấu trúc:

System: Bạn là SOC analyst. Phân tích log dưới đây và trả về JSON với schema: {verdict, severity, confidence, attack_type, attack_techniques, recommended_playbook, reasoning}

User: [LOG BATCH - tối đa 10 entries]

Log 1: {"timestamp":"...", "event":"auth_failed", "source_ip":"..."}

Log 2: {"timestamp": "...", "event": "auth_failed", "source_ip": "..."}

4.8.3. HITL Flow

```

async def handle_alert(result: AnalysisResult):
    if severity_rank(result.severity) >= severity_rank(ADMIN_APPROVAL_SEVERITY):
        alert_id = str(uuid4())
        pending_alerts[alert_id] = {
            "result": result,
            "created_at": time.time(),
            "status": "pending"
        }
        await log_pending_to_loki(result) # → Grafana Alert Rule (LogQL) phát hiện → gửi email cho admin
        await notify_admin(alert_id, result)
    else:
        await log_to_loki(result)

```

Endpoint SOAR webhook chỉ nhận request có SOAR_API_TOKEN hợp lệ trong header
 - tránh SOAR bị trigger từ bên ngoài.

[Hình 4.6 - Sơ đồ luồng HITL Approval]

(Chèn flowchart HITL)

4.9. SOAR Engine

4.9.1. Mapping Attack → Playbook

[Bảng 4.5 - Mapping attack type → playbook SOAR]

Attack Type	Target Service	Playbook
fraud_gate_bypass	payment-service	isolate_workload
lateral_movement	payment-service	isolate_workload
large_response	api-gateway	restrict_egress
cryptomining	api-gateway	quarantine_workload
port_scan	api-gateway	block_source_ip
exploit_probe	api-gateway	block_source_ip
brute_force	api-gateway	revoke_user_sessions
credential_stuffing	api-gateway	revoke_user_sessions
jwt_replay	api-gateway	revoke_user_sessions

4.9.2. Playbook Implementation - isolate_workload

```

async def isolate_workload(namespace: str, workload: str, context: str):
    k8s_client = load_kube_client(context)
    apps_v1 = client.AppsV1Api(k8s_client)
    core_v1 = client.CoreV1Api(k8s_client)

    # Patch Service selector → traffic không tới pod nào
    patch_body = {
        "spec": {
            "selector": {
                "app": workload,
                "ztlab-isolated": "true" # label không tồn tại trên pod
            }
        }
    }
    core_v1.patch_namespaced_service(
        name=workload, namespace=namespace, body=patch_body
    )

```

```
log_case(workload, "isolate_workload", original_selector)
```

4.9.3. RBAC Kubernetes cho SOAR

```
# k8s/rbac/soar-rbac.yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: soar-engine-role
rules:
  - apiGroups: ["apps"]
    resources: ["deployments"]
    verbs: ["get", "patch", "list"]
  - apiGroups: [""]
    resources: ["services"]
    verbs: ["get", "patch", "list"]
  - apiGroups: ["networking.k8s.io"]
    resources: ["networkpolicies"]
    verbs: ["get", "create", "patch", "list"]
```

4.10. Grafana Alerting — Cấu hình cảnh báo và Email

Grafana Alerting được cấu hình sẵn trong stack PLG trên namespace plg-stack, dùng Loki làm nguồn dữ liệu. Cấu hình gồm ba phần: kết nối SMTP để gửi email, các alert rule LogQL theo ngưỡng, và notification policy định tuyến theo severity.

Cấu hình SMTP qua Kubernetes Secret:

```
GF_SMTP_ENABLED      = "true"
GF_SMTP_HOST         = "smtp.gmail.com:587"
GF_SMTP_USER         = secretKeyRef(grafana-smtp/user)
GF_SMTP_PASSWORD     = secretKeyRef(grafana-smtp/password)
GF_SMTP_FROM_ADDRESS = "ztlab-alerts@ztlab.local"
```

Ví dụ alert rule LogQL theo ngưỡng:

```
# auth_failed > 10 trong 1 phút
sum(count_over_time({app="api-gateway"} |="auth_failed" [1m])) > 10
# fraud_blocked > 5 trong 1 phút
sum(count_over_time({app="fraud-detection"} |="fraud_blocked" [1m])) > 5
# opa_deny > 20 trong 2 phút
sum(count_over_time({app="opa"} |="decision=deny" [2m])) > 20
```

Notification policy định tuyến theo severity:

```
policies:
  - matchers: [ severity =~ "high|critical" ]
    receiver: email-admin
receivers:
  - name: email-admin      # SMTP contact point
    addresses: admin@ztlab.local
```

Luồng tích hợp:

log ứng dụng → Promtail → Loki → Grafana định kỳ poll và đánh giá alert rule (LogQL) → khi vượt ngưỡng, alert chuyển trạng thái Firing → notification policy định tuyến tới contact point SMTP → email cảnh báo được gửi tới quản trị viên.

CHƯƠNG 5: KIỂM THỬ VÀ ĐÁNH GIÁ

5.1. Phương pháp kiểm thử

5.1.1. Chiến lược kiểm thử

Nhóm áp dụng phương pháp **Black-box** và **Grey-box testing** kết hợp:

- **Functional Testing:** Kiểm tra luồng nghiệp vụ bình thường (happy path).
- **Security Testing:** 20 kịch bản tấn công từ scenario_01 đến scenario_20.
- **Performance Testing:** Đo overhead latency của lớp Zero Trust.
- **Integration Testing:** Kiểm tra end-to-end cross-cloud.

5.1.2. Môi trường kiểm thử

Máy kiểm thử: Laptop Ubuntu 22.04, kết nối qua SSH tunnel → Bastion → AWS cluster
KUBECONFIG: ~/.kube/ztlab/aws-tunnel.yaml
Context AWS: kubectl --context ctx-aws
Context OpenStack: kubectl --context ctx-openstack

Trạng thái hệ thống trước mỗi bộ test: database được seed bằng tests/seed_db.py với hai tài khoản ACC-1001 (1,000,000,000 VND) và ACC-2001 (250,000,000 VND).

5.2. Kiểm thử luồng nghiệp vụ bình thường

5.2.1. Đăng nhập và xác thực

(Kết quả kiểm thử - để trống chờ điền)

Test Case	Input	Expected	Actual	Pass/Fail
TC-01: Đăng nhập hợp lệ	testuser01/Test1234!	200 + session cookie		
TC-02: Sai mật khẩu	testuser01/wrongpass	401		
TC-03: Token hết hạn	JWT exp=past	401		
TC-04: Thiếu role	merchant01 → POST /payments	403		

5.2.2. Chuyển tiền bình thường

(Kết quả kiểm thử - để trống chờ điền)

Test Case	Giao dịch	Expected	Actual	Latency
TC-05: Chuyển tiền nhỏ	ACC-1001 → ACC-2001, 100,000 VND	completed, score=5		
TC-06: Chuyển tiền vừa	ACC-1001 → ACC-2001, 50,000,000 VND	completed, score=5		

Test Case	Giao dịch	Expected	Actual	Latency
TC-07: Xem số dư	GET /accounts/ACC-1001	200 + balance		
TC-08: Xem lịch sử	GET /transactions?account_id=ACC-1001	200 + list		

5.3. Kiểm thử các kịch bản tấn công

5.3.1. Scenario 01 - Brute Force / JWT Failure

Mô tả: Kẻ tấn công cố đăng nhập nhiều lần với mật khẩu sai để tìm credential hoặc bypass rate limit.

Cách thực hiện:

```
bash tests/scenario_01_brute_force.sh
# Gửi 100 request với token sai trong vòng 60 giây
```

Kết quả mong đợi:

- API Gateway từ chối sau 60 request/phút: 429 Too Many Requests.
- AI Analyzer phát hiện pattern 401 lặp từ cùng IP → alert brute_force, severity high.
- SOAR (sau approve): revoke_user_sessions.

[Hình 5.2 - Kết quả kịch bản Brute Force]

(Chèn screenshot: log trên Grafana, alert trên Web Portal, SOAR case)

Metric	Kết quả
Thời gian phát hiện (MTTD)	(đề trống)
Severity được gán	(đề trống)
Playbook đề xuất	revoke user sessions
Playbook thực thi	(đề trống sau approve)
Pass/Fail	(đề trống)

5.3.2. Scenario 02 - JWT Forgery

Mô tả: Kẻ tấn công cố tạo JWT giả mạo với algorithm none hoặc HS256 với key khác để bypass xác thực.

Cách thực hiện:

```
# tests/scenario_02_jwt_forgery.py
# Tạo JWT với algorithm="none" và role financial-write
forged_token = jwt.encode({"sub": "attacker",
    "realm_access": {"roles": ["financial-write"]}},
    "", algorithm="none")
```

Kết quả mong đợi: API Gateway reject với 401 invalid token; OPA deny nếu token đến được Envoy.

[Hình 5.3 - Kết quả kịch bản JWT Forgery]

(Chèn screenshot)

5.3.3. Scenario 03 - Lateral Movement / SVID Impersonation

Mô tả: Kẻ tấn công đã vào được một pod, cố gửi request trực tiếp đến payment-service mà không qua API Gateway, sử dụng SVID giả.

Kết quả mong đợi: OPA deny vì SVID không thuộc trust domain ztlab.local; AI Analyzer phát hiện pattern invalid svid → alert lateral_movement.

5.3.4. Scenario 04 - Fraud Gate Bypass

Mô tả: Cố gửi request trực tiếp đến Core Banking với header X-Fraud-Gate: passed giả mạo mà không qua fraud detection.

Cách thực hiện:

```
# tests/scenario_04_fraud_gate_bypass.py
requests.post(
    "http://127.0.0.1:30081/transactions/execute",
    headers={"X-Fraud-Gate": "passed", "X-Fraud-Score": "0"},
    json={...}
)
```

Kết quả mong đợi:

- OPA deny vì không có SVID hợp lệ từ trust domain.
- Core Banking cũng tự validate lại header - double validation.

5.3.5. Scenario 05 - High-Velocity Transaction

Mô tả: Gửi 50 giao dịch trong 60 giây từ cùng tài khoản để kiểm tra velocity detection.

Kết quả mong đợi: Fraud score tăng theo ngưỡng; từ giao dịch thứ 21+ nhận review; từ ngưỡng cao nhận block.

5.3.6. Scenario 06 - Data Exfiltration

Mô tả: Kẻ tấn công download lượng dữ liệu lớn bất thường từ API để staging data exfiltration.

Kết quả mong đợi: AI phát hiện bytes_sent lớn → alert large_response; SOAR restrict_egress.

[Hình 5.4 - Kết quả kịch bản Data Exfiltration]

(Chèn screenshot)

5.3.7. Tổng hợp kết quả 20 kịch bản

[Bảng 5.1 - Tổng hợp kết quả 20 kịch bản kiểm thử]

#	Kịch bản	Layer phát hiện	MTTD	Alert Severity	SOAR Playbook	Kết quả
01	Brute Force	API GW + AI	(trống)	high	revoke_user_sessions	(trống)
02	JWT Forgery	API GW	(trống)	high	block_source_ip	(trống)
03	Lateral Movement	Envoy/OPA + AI	(trống)	high	isolate_workload	(trống)
04	Fraud Gate Bypass	OPA + Core Banking	(trống)	critical	isolate_workload	(trống)
05	High Velocity	Fraud Detection	(trống)	medium	monitor_only	(trống)
06	Data Exfiltration	AI	(trống)	high	restrict_egress	(trống)
07	SVID Expiry	Envoy	(trống)	high	monitor_only	(trống)
08	Cross-Cloud Policy	OPA	(trống)	high	isolate_workload	(trống)
09	Privilege Escalation	AI	(trống)	critical	quarantine_workload	(trống)
10	Port Scan	AI	(trống)	medium	block_source_ip	(trống)
11	Cryptomining	AI	(trống)	high	quarantine_workload	(trống)
12	SOAR Response Validation	SOAR Engine	(trống)	-	-	(trống)
13	SQL Injection	Envoy/AI	(trống)	high	block_source_ip	(trống)
14	Command Injection	AI	(trống)	high	block_source_ip	(trống)
15	Account Manipulation	Core Banking	(trống)	high	isolate_workload	(trống)
16	Credential Stuffing	API GW + AI	(trống)	high	revoke_user_sessions	(trống)
17	Impair Defenses	AI	(trống)	critical	quarantine_workload	(trống)
18	Container Escape Signal	AI	(trống)	critical	quarantine_workload	(trống)
19	Data Staging	AI	(trống)	high	restrict_egress	(trống)
20	Replay Attack	API GW/Envoy	(trống)	high	revoke_user_sessions	(trống)

5.4. Đánh giá hiệu năng

5.4.1. MTTD và MTTR

[Bảng 5.2 - Số liệu MTTD và MTTR]

Metric	Mục tiêu	Kết quả đo được
MTTD - AI heuristic	< 3 phút	(trống)
MTTD - AI LLM (OpenAI)	< 5 phút	(trống)
MTTR - Sau admin approve	< 30 giây	(trống)
False Positive Rate	< 10%	(trống)
Alert Coverage (20 scenarios)	100%	(trống)

[Hình 5.6 - Biểu đồ phân phối thời gian phát hiện (MTTD)]

(Chèn biểu đồ box plot hoặc histogram)

5.4.2. Overhead hiệu năng ZTA

Đo latency của một request bình thường (POST /payments, 100,000 VND) trong hai điều kiện:

Baseline: Gọi trực tiếp payment-service (không qua Envoy/OPA).

ZTA Full: Gọi qua đầy đủ stack Envoy + OPA + mTLS.

[Hình 5.7 - Biểu đồ overhead hiệu năng ZTA]

(Chèn bar chart so sánh latency P50/P95/P99)

Percentile	Baseline	ZTA Full	Overhead
P50 (median)	(trống)	(trống)	(trống)
P95	(trống)	(trống)	(trống)
P99	(trống)	(trống)	(trống)

Dữ liệu thu thập bằng tests/perf_overhead.py với 1000 request mỗi điều kiện.

5.4.3. Chi phí tài nguyên

(Số liệu tài nguyên - để trống chờ điền)

Thành phần	CPU avg	Memory avg	Ghi chú
SPIRE Server	(trống)	(trống)	Tải CA, cấp SVID
OPA Server	(trống)	(trống)	Policy evaluation
Envoy Sidecar (per pod)	(trống)	(trống)	Inbound + outbound
AI Analyzer	(trống)	(trống)	Heuristic mode
Loki	(trống)	(trống)	10Gi PVC

5.5. So sánh với các giải pháp hiện có

[Bảng 5.3 - So sánh ZTLab với các giải pháp hiện có]

Tiêu chí	ZTLab	Cloudflare One	Zscaler ZIA	Palo Alto Prisma
Zero Trust ZTA	Có (đầy đủ)	Có (đầy đủ)	Có (đầy đủ)	Có (đầy đủ)
Workload Identity (SVID)	Có - SPIFFE/SPIRE	Không	Không	Một phần
Policy as Code	Có - OPA Rego	Không (GUI)	Không (GUI)	Không (GUI)
AI-driven Detection	Có - LLM + Heuristic	ML-based	ML-based	ML-based
HITL trước SOAR	Có (bắt buộc)	Không	Tùy chọn	Tùy chọn
Multi-Cloud K8s native	Có - K3s cross-cluster	Một phần	Không	Một phần
Mã nguồn mở	Có (100%)	Không (SaaS)	Không (SaaS)	Không (SaaS)
Chi phí (lab scale)	~\$50/tháng	~\$500/tháng	~\$800/tháng	~\$1000+/tháng

Tiêu chí	ZTLab	Cloudflare One	Zscaler ZIA	Palo Alto Prisma
Rollback playbook	Có	Không	Không	Một phần

CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Kết luận

Chương này tổng kết lại toàn bộ quá trình nghiên cứu, thiết kế và triển khai hệ thống đã được trình bày trong các chương trước, đối chiếu kết quả đạt được với các mục tiêu đã đề ra ở Chương 1. Bên cạnh việc khẳng định những đóng góp về mặt khoa học và kỹ thuật, chương cũng nhìn nhận một cách thẳng thắn những hạn chế còn tồn tại của giải pháp, từ đó đề xuất một lộ trình phát triển có tính khả thi nhằm hoàn thiện hệ thống trong tương lai.

6.1.1. Tóm tắt đóng góp

Đề tài đã nghiên cứu, thiết kế và triển khai thành công một hệ thống bảo mật vận hành theo mô hình Zero Trust, tích hợp đồng bộ ba trụ cột SIEM – SOAR – AI trên một môi trường Multi-Cloud thực tế bao gồm Amazon Web Services (AWS) và OpenStack. Khác với phần lớn các nghiên cứu trước đây vốn chỉ dừng ở mức mô phỏng hoặc kiểm chứng từng thành phần một cách riêng lẻ, hệ thống ZTLab trong đề tài này được hiện thực hóa thành một kiến trúc hoàn chỉnh, có khả năng vận hành xuyên suốt (end-to-end) từ yêu cầu của người dùng cho tới tầng cơ sở dữ liệu nằm trên một nhà cung cấp đám mây khác. Những đóng góp chính của đề tài có thể được phân thành hai nhóm: đóng góp về mặt kỹ thuật và đóng góp về mặt kết quả kiểm thử thực nghiệm.

a) Về mặt kỹ thuật

Thứ nhất, kiến trúc Zero Trust end-to-end cho Kubernetes Multi-Cloud. Hệ thống đã tích hợp một cách nhất quán bốn thành phần định danh và thực thi chính sách thành một thể thống nhất: SPIFFE/SPIRE đảm nhận việc cấp phát định danh cho workload (SVID) dựa trên cơ chế chứng thực (attestation) thay vì dựa vào địa chỉ mạng; Envoy đóng vai trò Điểm thực thi chính sách (Policy Enforcement Point – PEP), chặn và kiểm tra mọi kết nối ngay tại tầng sidecar; Open Policy Agent (OPA) đóng vai trò Điểm quyết định chính sách (Policy Decision Point – PDP) với chính sách được biểu diễn dưới dạng mã (policy-as-code) bằng ngôn ngữ Rego; và Keycloak quản lý định danh của người dùng cuối. Sự kết hợp này loại bỏ hoàn toàn khái niệm “tin cậy ngầm định” (implicit trust) vốn là gốc rễ của nhiều cuộc tấn công di chuyển ngang (lateral movement): mỗi yêu cầu đều phải được xác thực, ủy quyền và mã hóa độc lập, bất kể nó xuất phát từ bên trong hay bên ngoài vành đai mạng.

Thứ hai, luồng thanh toán cross-cloud được bảo vệ theo chiều sâu. Đường đi của một giao dịch từ trình duyệt người dùng cho tới cơ sở dữ liệu đặt trên OpenStack được bảo vệ bởi ba lớp gateway độc lập – Infrastructure Gateway, API Gateway và lớp Envoy/OPA – trong đó mỗi lớp đảm nhận một chức năng kiểm soát riêng biệt và sử dụng một cơ chế tin cậy khác nhau (xác thực hạ tầng, xác thực JWT của người dùng, và xác thực SVID của workload). Thiết kế phòng thủ theo chiều sâu (defense-in-depth) này bảo đảm rằng việc một lớp bị vượt qua không đồng nghĩa với việc toàn bộ luồng bị xâm phạm, đồng thời không lớp nào trở thành điểm lỗi đơn (single point of failure) của hệ thống.

Thứ ba, hệ thống giám sát an ninh tập trung cho môi trường Multi-Cloud. Ngăn xếp PLG (Promtail – Loki – Grafana) được triển khai theo kiến trúc relay, cho phép Promtail chạy trên phía OpenStack đẩy nhật ký (log) về Loki đặt trên phía AWS, qua đó tạo ra một điểm tập trung log duy nhất cho toàn bộ hệ thống phân tán. Việc gắn một định danh truy vết (trace_id) xuyên suốt mọi chặng của giao dịch giúp khắc phục bài toán phân mảnh log – một thách thức cố hữu của các kiến trúc microservices trên nhiều đám mây – và là tiền đề bắt buộc để khâu phân tích và phản ứng tự động hoạt động chính xác. Bên cạnh đó, Grafana Alerting cung cấp một kênh cảnh báo theo ngưỡng (threshold-based) chạy song song với AI Analyzer: các alert rule LogQL phát hiện vượt ngưỡng và gửi thông báo email tới quản trị viên qua SMTP contact point, giúp hệ thống chủ động cảnh báo mà không phụ thuộc vào công cụ quản lý sự cố bên ngoài.

Thứ tư, chuyên viên phân tích an ninh (SOC Analyst) tự động dựa trên AI. Thành phần AI Analyzer được thiết kế với một chuỗi nhà cung cấp dự phòng (provider chain) theo thứ tự OpenAI → Gemini → Heuristic, bảo đảm rằng khâu phát hiện không bao giờ rơi vào trạng thái ngừng hoạt động ngay cả khi các dịch vụ mô hình ngôn ngữ lớn bên ngoài không khả dụng. Với mỗi cảnh báo, hệ thống tự động phân loại mức độ nghiêm trọng, ánh xạ sang kỹ thuật tương ứng trong ma trận MITRE ATT&CK và đề xuất playbook phản ứng phù hợp, qua đó giảm đáng kể khối lượng công việc thủ công của đội ngũ vận hành an ninh.

Thứ năm, cơ chế phản ứng tự động SOAR với Human-in-the-Loop bắt buộc. Mọi hành động có khả năng tác động tới môi trường production – chẳng hạn cô lập workload, thu hồi phiên người dùng hay chặn địa chỉ nguồn – đều bắt buộc phải qua bước phê duyệt của quản trị viên trước khi được thực thi. Mỗi playbook đều được ghi lại trong một nhật ký kiểm toán (audit trail) bền vững và được trang bị cơ chế hoàn tác (rollback), nhờ đó hệ thống cân bằng được giữa tốc độ phản ứng tự động và yêu cầu kiểm soát rủi ro vận hành trong một môi trường tài chính nhạy cảm.

b) Về mặt kết quả kiểm thử thực nghiệm

Hệ thống đã được kiểm chứng thông qua bộ 20 kịch bản tấn công bao phủ nhiều chiến thuật và kỹ thuật trong ma trận MITRE ATT&CK, từ tấn công vét cạn (brute force), giả mạo JWT, di chuyển ngang, cho tới đánh cắp và rò rỉ dữ liệu. Các chỉ số định lượng chính được tổng hợp như sau: tỷ lệ phát hiện đạt [điền: ...]% trên tổng số 20 kịch bản; thời gian phát hiện trung bình (MTTD) là [điền: ...] phút; thời gian phản ứng trung bình (MTTR) tính từ thời điểm quản trị viên phê duyệt là [điền: ...] giây; và độ trễ tăng thêm (overhead) do lớp Zero Trust gây ra, đo tại bách phân vị P99, là [điền: ...] ms. Chi tiết của các số liệu này được trình bày đầy đủ trong Bảng 5.1, Bảng 5.2 và Mục 5.4. Các giá trị còn để trống ở trên sẽ được điền sau khi hoàn tất quá trình thu thập số liệu đo lường thực tế.

6.1.2. Hạn chế

Bên cạnh những kết quả đạt được, đề tài cũng tồn tại một số hạn chế cần được nhìn nhận một cách khách quan, vừa để bảo đảm tính trung thực khoa học, vừa làm cơ sở định hướng cho các nghiên cứu tiếp theo.

Về quy mô và tải thực nghiệm. Toàn bộ hệ thống được kiểm chứng trong môi trường phòng thí nghiệm (lab) với lưu lượng truy cập mô phỏng ở mức thấp, do đó chưa phản ánh đầy đủ các đặc tính của một hệ thống production thực tế. Các chỉ số về độ trễ, độ ổn định và khả năng chịu lỗi cần được kiểm chứng lại dưới mức tải cao hơn nhiều – chẳng hạn khoảng 1.000 yêu cầu mỗi giây (RPS) – để đánh giá chính xác hành vi của lớp Zero Trust khi hệ thống đạt ngưỡng bão hòa.

Về cơ chế chứng thực của SPIRE trên OpenStack. Trong cấu hình hiện tại, join_token dùng để đăng ký SPIRE Agent trên phía OpenStack phải được tái tạo mỗi khi máy ảo (VM) khởi động lại, gây bất tiện cho vận hành và làm giảm tính tự động hóa của hệ thống. Đây là hệ quả của việc sử dụng phương thức chứng thực dựa trên token tĩnh; hạn chế này cần được khắc phục bằng các phương thức node attestation mạnh hơn và có khả năng tự phục hồi, ví dụ aws_iid trên phía AWS hoặc một attestor chuyên dụng cho OpenStack.

Về độ chính xác của AI Analyzer ở chế độ dự phòng. Khi hoạt động ở chế độ heuristic – tức là khi cả OpenAI lẫn Gemini đều không khả dụng – AI Analyzer có tỷ lệ bỏ sót (false negative) cao hơn so với khi sử dụng mô hình ngôn ngữ lớn, do các luật heuristic chỉ bao phủ được những mẫu tấn công đã biết. Việc nâng cao độ chính xác của chế độ này đòi hỏi một tập dữ liệu được gán nhãn (labeled dataset) đủ lớn và đại diện để tinh chỉnh (fine-tune) các ngưỡng phát hiện.

Về năng lực phát hiện của cơ chế cảnh báo. Grafana Alerting chỉ phát hiện bất thường dựa trên ngưỡng khối lượng (threshold-based volume) thông qua truy vấn LogQL, do đó không phân tích được ngữ nghĩa hay tương quan hành vi phức tạp như một nền tảng điều tra sự cố chuyên dụng. Việc nâng cao năng lực này đòi hỏi

bổ sung phân tích tương quan đa nguồn và làm giàu ngữ cảnh (enrichment) cho mỗi cảnh báo.

6.2. Hướng phát triển

Trên cơ sở những kết quả và hạn chế đã phân tích, đề tài đề xuất một lộ trình phát triển gồm ba giai đoạn – ngắn hạn, trung hạn và dài hạn – được sắp xếp theo mức độ ưu tiên và độ phức tạp kỹ thuật tăng dần. Lộ trình này hướng tới mục tiêu đưa hệ thống từ trạng thái nguyên mẫu (prototype) trong phòng thí nghiệm tiến gần hơn tới một giải pháp có thể triển khai ở quy mô doanh nghiệp.

6.2.1. Ngắn hạn (3–6 tháng)

Giai đoạn ngắn hạn tập trung vào việc khắc phục trực tiếp các khoảng trống đã được chỉ ra ở Mục 6.1.2, đồng thời củng cố độ hoàn thiện của các thành phần đã có.

Mở rộng phạm vi bảo vệ của Envoy sidecar. Bổ sung Envoy sidecar cho account-service và transaction-service trên phía OpenStack – hai dịch vụ hiện chưa được trang bị sidecar, tạo thành một khoảng trống trong mô hình Zero Trust. Việc này giúp bảo đảm rằng mọi kết nối nội bộ giữa các dịch vụ tài chính đều được kiểm tra và thực thi chính sách một cách nhất quán.

Tối ưu Grafana Alert Rules với dynamic threshold. Thay thế các ngưỡng tĩnh hiện tại bằng cơ chế ngưỡng động (dynamic threshold) tự động điều chỉnh theo đường cơ sở (baseline) lưu lượng và hành vi bình thường của từng dịch vụ, ví dụ dựa trên độ lệch chuẩn hoặc trung bình trượt. Cách tiếp cận này giảm cảnh báo giả (false positive) trong giờ cao điểm và tăng độ nhạy khi tải thấp.

Nâng cấp AI Analyzer bằng kỹ thuật RAG. Áp dụng Retrieval-Augmented Generation với một kho vector (vector store) lưu trữ các báo cáo sự cố trước đây, cho phép mô hình đối chiếu mẫu tấn công mới với lịch sử đã xử lý. Cách tiếp cận này vừa tăng độ chính xác của phân tích, vừa tạo ra tính nhất quán trong các khuyến nghị phản ứng theo thời gian.

Bổ sung cơ chế chứng thực cho SPIRE trên OpenStack. Thay thế phương thức join_token tĩnh bằng một attestor mạnh hơn – ví dụ dựa trên IAS (Intel Attestation Service) hoặc một attestor tùy biến cho OpenStack – nhằm loại bỏ thao tác tái tạo token thủ công và nâng cao mức độ tin cậy của định danh workload.

6.2.2. Trung hạn (6–12 tháng)

Giai đoạn trung hạn hướng tới việc nâng cấp về mặt kiến trúc, chuyển từ các thành phần được triển khai và quản lý thủ công sang những nền tảng có khả năng quản trị tập trung và vận hành ở quy mô lớn hơn.

Triển khai Istio Service Mesh. Thay thế mô hình Envoy sidecar độc lập bằng Istio nhằm quản lý chính sách một cách tập trung và nhất quán hơn, đồng thời tận dụng các tính năng nâng cao như điều hướng lưu lượng (traffic shifting) và ngắt mạch (circuit breaker). Việc này giảm gánh nặng cấu hình thủ công cho từng sidecar và tạo nền tảng cho các chiến lược triển khai an toàn như canary release.

Liên kết đa cụm với Cilium. Áp dụng Cilium để hiện thực hóa NetworkPolicy xuyên cụm (across clusters) dựa trên công nghệ eBPF, thay thế cho việc định tuyến WireGuard thủ công hiện tại. Cách tiếp cận này mang lại khả năng quan sát (observability) ở tầng nhân hệ điều hành và một mô hình thực thi chính sách mạng có hiệu năng cao hơn.

Tự động hóa phản ứng cho các kịch bản rủi ro thấp. Mở rộng cơ chế SOAR cho phép tự động xử lý (auto-remediation) mà không cần phê duyệt thủ công đối với các kịch bản rủi ro thấp, dựa trên điểm tin cậy (confidence score) và lịch sử xử lý. Cụ thể, khi độ tin cậy của AI vượt 0,95 và loại tấn công đã được ghi nhận trên 10 lần, hệ thống có thể tự phê duyệt – qua đó rút ngắn MTTR cho những tình huống đã được kiểm chứng, trong khi vẫn giữ Human-in-the-Loop cho các trường hợp nghiêm trọng hoặc bất thường.

Tự động sinh báo cáo tuân thủ. Xây dựng năng lực tự động tạo lập các báo cáo tuân thủ theo những khung tiêu chuẩn như PCI-DSS và ISO 27001, dựa trên nhật ký kiểm toán của SOAR và nhật ký quyết định (decision log) của OPA. Việc này biến dữ liệu vận hành an ninh thành bằng chứng tuân thủ có thể kiểm toán, giảm đáng kể công sức chuẩn bị cho các kỳ đánh giá.

6.2.3. Dài hạn (12+ tháng)

Giai đoạn dài hạn mang tính nghiên cứu và mở rộng, nhằm kiểm chứng tính tổng quát của kiến trúc cũng như khám phá những hướng tiếp cận tiên tiến trong lĩnh vực an ninh chủ động.

Mở rộng sang Google Cloud và Azure. Triển khai hệ thống trên ba nhà cung cấp đám mây trở lên nhằm kiểm chứng tính tổng quát (generality) và khả năng di động (portability) của kiến trúc Zero Trust đã đề xuất, qua đó khẳng định rằng giải pháp không bị ràng buộc vào một nền tảng đám mây cụ thể.

Xây dựng Digital Twin phục vụ kiểm thử an ninh. Phát triển một bản sao số (digital twin) của hệ thống trong môi trường sandbox để thử nghiệm các playbook phản ứng trước khi áp dụng vào production. Cách làm này cho phép đánh giá tác động của một hành động phản ứng mà không gây rủi ro cho hệ thống thực, nâng cao độ an toàn của quá trình tự động hóa.

Nghiên cứu Federated Learning cho tình báo môi đe dọa. Ứng dụng học liên kết (federated learning) để chia sẻ mô hình phát hiện mối đe dọa giữa nhiều tổ chức mà

không cần chia sẻ nhật ký thô, qua đó vừa nâng cao năng lực phát hiện chung của cộng đồng, vừa bảo toàn tính riêng tư của dữ liệu (privacy-preserving threat intelligence).

TÀI LIỆU THAM KHẢO

- [1] Gartner, Inc. (2024). *Gartner Forecast: Cloud Services, Worldwide*. Gartner Research.
- [2] IBM Security. (2023). *Cost of a Data Breach Report 2023*. IBM Corporation. <https://www.ibm.com/reports/data-breach>
- [3] Rose, S., Borchert, O., Mitchell, S., & Connelly, S. (2020). *Zero Trust Architecture* (NIST Special Publication 800-207). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.SP.800-207>
- [4] Cloud Native Computing Foundation. (2023). *SPIFFE: Secure Production Identity Framework for Everyone*. CNCF. <https://spiffe.io/>
- [5] Styra, Inc. (2023). *Open Policy Agent Documentation*. <https://www.openpolicyagent.org/docs/latest/>
- [6] Envoy Proxy Authors. (2023). *Envoy Proxy Documentation*. <https://www.envoyproxy.io/docs/envoy/latest/>
- [7] Red Hat, Inc. (2023). *Keycloak Documentation*. <https://www.keycloak.org/documentation>
- [8] Grafana Labs. (2023). *Grafana Alerting Documentation*. <https://grafana.com/docs/grafana/latest/alerting/>
- [9] MITRE Corporation. (2023). *MITRE ATT&CK® Framework*. <https://attack.mitre.org/>
- [10] Kubernetes Authors. (2023). *Kubernetes Documentation*. <https://kubernetes.io/docs/>
- [11] Rancher Labs. (2023). *K3s: Lightweight Kubernetes*. <https://k3s.io/>
- [12] WireGuard Authors. (2023). *WireGuard: Fast, Modern, Secure VPN Tunnel*. <https://www.wireguard.com/>
- [13] HashiCorp. (2023). *Terraform Documentation*. <https://developer.hashicorp.com/terraform/docs>
- [14] Red Hat Ansible. (2023). *Ansible Documentation*. <https://docs.ansible.com/>

- [15] Grafana Labs. (2023). *Loki Documentation*. <https://grafana.com/docs/loki/latest/>
- [16] OpenAI. (2024). *GPT-4 Technical Report*. <https://openai.com/research/gpt-4>
- [17] Google DeepMind. (2024). *Gemini: A Family of Highly Capable Multimodal Models*. <https://deepmind.google/technologies/gemini/>
- [18] Kindervag, J. (2010). *Build Security Into Your Network's DNA: The Zero Trust Network Architecture*. Forrester Research.
- [19] Patel, R., & Shah, M. (2022). Zero Trust Security in Cloud-Native Environments: A Systematic Review. *IEEE Access*, 10, 12345–12360.
- [20] Nguyen, T. H., & Tran, V. L. (2023). SOAR Integration with AI for Automated Incident Response in Multi-Cloud Environments. *Journal of Cybersecurity Research*, 8(2), 45–62.

PHỤ LỤC

Phụ lục A - Cấu hình môi trường

A.1. Yêu cầu cài đặt

Hệ thống yêu cầu các công cụ và phiên bản tối thiểu sau đây trên máy điều khiển (control node) trước khi tiến hành triển khai:

```
# Công cụ cần thiết
kubectl    >= 1.28
ansible    >= 2.14
terraform  >= 1.5
docker     >= 24.0
python     >= 3.11
wireguard-tools
```

A.2. Biến môi trường

Các biến môi trường chính điều khiển hành vi của hệ thống được liệt kê trong bảng dưới đây. Các giá trị nhạy cảm (khóa API, mật khẩu, token) không được lưu trực tiếp trong mã nguồn mà phải được cung cấp qua tệp cấu hình bí mật.

Biến	Giá trị mặc định	Mô tả
AI_PROVIDER	heuristic	Provider AI: heuristic, openai, gemini
OPENAI_API_KEY	(trống)	API key OpenAI (không commit vào Git)

Biến	Giá trị mặc định	Mô tả
GEMINI_API_KEY	(trống)	API key Google Gemini
SOAR_DRY_RUN	true	Dry-run mode cho SOAR
SOAR_MIN_SEVERITY	high	Ngưỡng severity tối thiểu để SOAR act
AI_ANALYZER_ADMIN_APPROVAL_SEVERITY	high	Ngưỡng yêu cầu admin approve
KEYCLOAK_ADMIN_PASSWORD	-	Không có default, phải set
SOAR_API_TOKEN	(trống)	Token xác thực SOAR webhook

A.3. Khởi động nhanh

Quy trình triển khai toàn bộ hệ thống từ đầu được thực hiện theo các bước tuần tự sau:

1. Clone và cấu hình

```
git clone <repo>
```

```
cp .env.template .env && cp .env.ai.template .env.ai
```

Điền các giá trị trong .env và .env.ai

2. Provision hạ tầng

```
cd terraform/aws && terraform init && terraform apply
```

```
cd terraform/openstack && terraform init && terraform apply
```

3. Cấu hình nodes

```
ansible-playbook -i ansible/inventory/hosts.yml ansible/baseline.yml
```

```
ansible-playbook -i ansible/inventory/hosts.yml ansible/wireguard.yml
```

4. Mở tunnel Kubernetes

```
bash scripts/k8s-tunnel.sh up all
```

```
export KUBECONFIG=~/.kube/ztlab/aws-tunnel.yaml
```

5. Deploy toàn bộ stack

```
bash scripts/deploy-all.sh
```

6. Mở UI tunnel (terminal riêng)

```
ssh -N -i ~/.ssh/zta-siem-soar-key \
```

```
-L 8080:10.10.1.10:80 \
```

```
-J ubuntu@54.254.145.86 ubuntu@10.10.1.10
```

Phụ lục B - Danh sách kịch bản kiểm thử

Bộ kịch bản kiểm thử gồm 20 kịch bản tấn công, mỗi kịch bản được hiện thực hóa thành một tệp script độc lập và được ánh xạ tới kỹ thuật tương ứng trong ma trận MITRE ATT&CK, tạo điều kiện cho việc tái lập (reproducibility) và đối chiếu kết quả.

File	Kịch bản	MITRE Technique
scenario_00_full_suite.py	Chạy toàn bộ suite	-
scenario_01_brute_force.sh	Brute force đăng nhập	T1110.001
scenario_02_jwt_forgery.py	Giả mạo JWT	T1550.001
scenario_03_lateral_movement.sh	Di chuyển ngang với SVID sai	T1550.001
scenario_04_fraud_gate_bypass.py	Bypass fraud gate	T1562
scenario_05_high_velocity.py	Giao dịch tốc độ cao	T1078
scenario_06_exfiltration.py	Đánh cắp dữ liệu	T1030
scenario_07_svid_expiry.sh	SPIRE/SVID hết hạn	T1553
scenario_08_cross_cloud.sh	Kiểm tra policy cross-cloud	T1021
scenario_09_privesc.sh	Leo thang đặc quyền	T1068
scenario_10_portscan.sh	Quét cổng	T1046
scenario_11_cryptomining.sh	Khai thác tiền mã hóa	T1496
scenario_12_soar_response.sh	Xác nhận phản ứng SOAR	-
scenario_13_sql_injection.sh	SQL injection	T1190
scenario_14_command_injection.sh	Command injection	T1059
scenario_15_account_manipulation.sh	Thao túng tài khoản	T1098
scenario_16_credential_stuffing.sh	Nhồi credential	T1078.003
scenario_17_impair_defenses.sh	Vô hiệu hóa phòng thủ	T1562
scenario_18_container_escape.sh	Thoát container	T1611
scenario_19_data_staging.sh	Thu thập dữ liệu	T1074
scenario_20_replay_attack.sh	Tấn công replay	T1539

Phụ lục C - Prometheus Metrics

Các chỉ số (metrics) tùy chỉnh phục vụ giám sát được định nghĩa tập trung trong tệp `shared/metrics.py` và được Prometheus thu thập theo mô hình pull. Tập nhãn (label) đi kèm mỗi metric cho phép phân tách dữ liệu theo dịch vụ, đám mây và trạng thái giao dịch.

```
# Tổng giao dịch theo trạng thái
ztlab_transactions_total{service, cloud, type, status}

# Phân phối fraud score theo verdict
ztlab_fraud_score{service, cloud, verdict}

# Latency cross-cloud
ztlab_cross_cloud_latency_seconds{source, target, status}

# Tổng lỗi xác thực
ztlab_auth_failures_total{service, cloud, reason}

# Trạng thái service
ztlab_service_up{service, cloud}
```

Phụ lục D - Grafana Alert Rules và Email cảnh báo

[Hình 5.8 - Email cảnh báo mẫu do Grafana Alerting gửi] (chèn ảnh chụp email cảnh báo do Grafana gửi qua SMTP)

Mỗi alert rule LogQL khi chuyển sang trạng thái Firing sẽ kích hoạt notification policy, gửi email cảnh báo tới quản trị viên qua SMTP contact point. Một cấu hình alert rule tiêu biểu (định dạng provisioning YAML) và email cảnh báo mẫu có dạng như sau:

```
# Grafana Alert Rule (provisioning YAML)
groups:
- name: ztlab-alerts
  folder: ZTLab
  rules:
- uid: high-auth-failures
  title: HighAuthFailures
  condition: A
  data:
- refId: A
  datasourceUid: loki
  model:
    expr: sum(count_over_time({app="api-gateway"} |= "auth_failed" [1m]))
    for: 1m
  labels: { severity: high, service: api-gateway }
  annotations:
    summary: "Auth failures vượt ngưỡng (>10/phút)"
```

```
# Email cảnh báo mẫu (Grafana → SMTP contact point)
From: ztlab-alerts@ztlab.local
To: admin@ztlab.local
Subject: [FIRING:1] HighAuthFailures (high api-gateway)
```

```
Alert HighAuthFailures đang ở trạng thái FIRING
Severity: high
Service: api-gateway
Value: 47 (ngưỡng > 10/phút)
Time: 2026-06-22 14:30:05
Description: Phát hiện 47 lần auth_failed từ IP 10.0.0.99 trong 2 phút.
```

Phụ lục E - Luồng xử lý POST /payments chi tiết

Sơ đồ tuần tự (sequence diagram) dưới đây mô tả đầy đủ đường đi của một giao dịch thanh toán hợp lệ, kèm theo mốc thời gian ước lượng tại từng bước, từ thời điểm trình duyệt gửi yêu cầu cho tới khi nhận được phản hồi hoàn tất. Sơ đồ làm rõ vị trí của từng điểm kiểm soát Zero Trust (API Gateway, Envoy, OPA, fraud gate) cũng như chặng truyền cross-cloud qua đường hầm WireGuard.

```
t=0ms Browser POST /payments (JWT Bearer)
t=1ms Traefik Ingress route → api-gateway:8080
t=5ms API Gateway: rate limit check PASS
t=12ms API Gateway: JWKS fetch (cached) + JWT verify PASS
```

t=13ms API Gateway: role check financial-write PASS
t=14ms API Gateway: tạo X-Trace-ID, forward → payment-service via K8s Service
t=15ms Envoy inbound (payment-service): intercept tại port 15006
t=16ms Envoy → OPA gRPC CheckRequest
t=20ms OPA eval zta_policy.rego → allow=true (Bearer + role OK)
t=21ms Envoy forward → payment-service app port 8080
t=22ms payment-service: parse body, gọi fraud-detection /score
t=30ms Envoy (fraud-detection) → OPA → allow (SVID payment-service)
t=35ms fraud-detection: Redis velocity check + score calc → 5
t=36ms fraud-detection: return {"score":5, "verdict":"allow", "gate":"passed"}
t=37ms payment-service: gọi core-banking /transactions/execute
t=38ms Envoy outbound route /transactions → cluster core_banking (10.10.1.12:30081)
t=40ms [WireGuard tunnel ~2ms RTT]
t=42ms Envoy inbound (core-banking OpenStack): intercept
t=43ms Envoy → OPA: check SVID + X-Fraud-Gate + X-Fraud-Score
t=47ms OPA: allow=true (SVID ztlab.local + fraud gate passed)
t=48ms core-banking: gọi account-service /accounts/transfer
t=55ms PostgreSQL: SELECT FOR UPDATE + UPDATE balance
t=60ms core-banking: gọi transaction-service /transactions (ghi ledger)
t=65ms PostgreSQL: INSERT ledger record
t=66ms core-banking → payment-service: {status:"completed", ...}
t=68ms [WireGuard return]
t=70ms payment-service → notification-service /notify (async, best-effort)
t=72ms payment-service → API Gateway: response
t=73ms API Gateway → Browser: {status:"completed", trace_id:..., fraud:{...}, core_banking:{...}}

Báo cáo này được thực hiện bởi sinh viên Hoàng Bảo Phước (23521231) và Phạm Võ Khánh Hà (23520414), Khoa Mạng Máy tính và Truyền thông, Trường Đại học Công nghệ Thông tin - ĐHQG TP.HCM, dưới sự hướng dẫn của ThS. Đỗ Thị Phương Uyên.

TP. Hồ Chí Minh, tháng 6 năm 2026